

G

READ-ONLY REPOSITORY AUDIT · 17 AUGUST 2026

GoHa Full System Report

A detailed product, architecture, database, security, quality, operations, and repository reference for the complete current working tree.

12

PRIMARY MODULES

19

DATABASE TABLES

62

SERVER ACTIONS

358

TRACKED FILES

System snapshot

Local branch `main`, working tree as inspected before report creation. Next.js 16.2.10, React 19.2.4, PostgreSQL/Neon, Drizzle, Better Auth.

Audit boundary

Static/read-only inspection. No live database connection, migrations, account scripts, build, unit test, E2E test, or production request was executed.

Scope, method, and evidence

What was and was not inspected, and how to interpret the conclusions.

Read-only promise honored

Application source, configuration, Git metadata, schema/migrations, existing generated artifacts, and saved QA images were inspected without changing the application. No live database state was queried and no test data was created, reset, or deleted. The only intended workspace output is this PDF; the temporary report generator is removed after validation.

Coverage

- Every App Router page, layout, route boundary, Route Handler, and Server Action under app/.
- All UI components, shared primitives, shell/navigation, charts, responsive behavior, theme, motion, and Focus client store.
- Every schema, enum, relation, repository, validation module, date/derivation helper, migration SQL file, and migration snapshot/journal state.
- Authentication/session flow, route protection, owner scoping, archive/delete lifecycle, export, privacy-sensitive scripts, and environment contract.
- Package/dependency configuration, TypeScript, ESLint, Next, Tailwind, Drizzle, Vitest, Playwright, Git hygiene, docs, design references, generated Graphify output, and existing build/test/screenshot artifacts.
- Tracked and relevant untracked files. node_modules, .next compilation internals, .git object storage, secret values, and live external infrastructure were not line-by-line audited as product source.

Evidence levels

LABEL	MEANING
CONFIRMED	Directly visible in source, schema, configuration, Git metadata, or an existing local artifact.
KNOWN ISSUE	Explicitly documented by the repository itself and also consistent with the implementation.
RISK	A plausible failure path demonstrated by missing constraints/guards or non-atomic sequencing; occurrence was not triggered.
NOT VERIFIED	Requires live database, hosted deployment, runtime telemetry, or a fresh execution that this read-only audit intentionally avoided.

Important limitations

- The audit cannot prove which migrations are applied to Neon, current row counts, actual backups/PITR configuration, current users/sessions, production headers, production TLS, hosted environment variables, or serverless topology.

- Historical build and test claims are reported as historical evidence, not as fresh validation. Existing screenshots may contain test-state/transitional UI and were not regenerated.
- Environment keys and structural validity were considered where already visible locally; secret values are neither reproduced nor included in this report.
- A complete source sweep does not mean every external dependency implementation was audited. Dependency behavior is described only where configured or verified from the installed local version.

Contents

The report is designed as both an audit and a maintainable technical reference.

03	Executive assessment	14	Export, archive, deletion, privacy
04	Product model and lifecycle	15	Design, responsive UI, accessibility
05	Architecture and request flow	16	Testing and quality evidence
06	Technology and repository state	17	Build, deployment, and operations
07	Route and shell catalog	18	Documentation and repository hygiene
08	Feature-by-feature reference	19	Prioritized risk register
09	Cross-feature consistency	20	Remediation roadmap
10	Data model: 19 tables	A	Server Action catalog
11	Migrations and schema evolution	B	Complete file inventory
12	Authentication and security	C	Glossary and ownership map
13	Validation, transactions, caching		

03

Executive assessment

GoHa is a substantial, coherent single-owner productivity system. Its principal gap is the transition from a strong local product to a repeatable, guarded, recoverable production operation.

Overall conclusion

The application is much more than a dashboard prototype. It has a disciplined server-first architecture, a connected domain model, meaningful timezone-aware derivations, reusable UI primitives, 12 working modules, 19 tables, 11 migrations, 62 Server Actions, and broad historical browser coverage. The strongest product quality is canonical integration: tasks, habits, focus, goals, reviews, progress, and calendar generally read the same durable rows rather than duplicating truth.

The most material weaknesses are integrity and operability boundaries: an untracked required migration, destructive test tooling without a test-database guard, two legacy migration pairs that can lose meaning, an export labeled as complete that is not a backup, several multi-statement writes without transactions, and no CI/deployment/backup/observability runbook. Product correctness also has four high-confidence cross-screen defects: Review week state, numeric habit outcomes, Calendar data windows/timezone, and rapid Brain Dump capture.

What is strong

Product coherence

- Clear Life Area → Goal → Task/Habit → Today → Focus → Review/Progress loop.
- Canonical records drive multiple modules.
- Archive/restore, undo, empty/error/loading states, and explainable recommendations.

Engineering foundation

- Strict TypeScript and Next lint presets.
- Server-only database boundary and session-derived identity.
- Typed schema/repositories, Zod validation, parameterized SQL conversion.

Interaction quality

- Responsive shell, light/dark themes, reduced motion/opacity fallbacks.
- Shared focus-managed modal/detail panel, keyboard-capable controls.
- Timestamp-derived Focus timer survives reload/sleep.

Historical QA

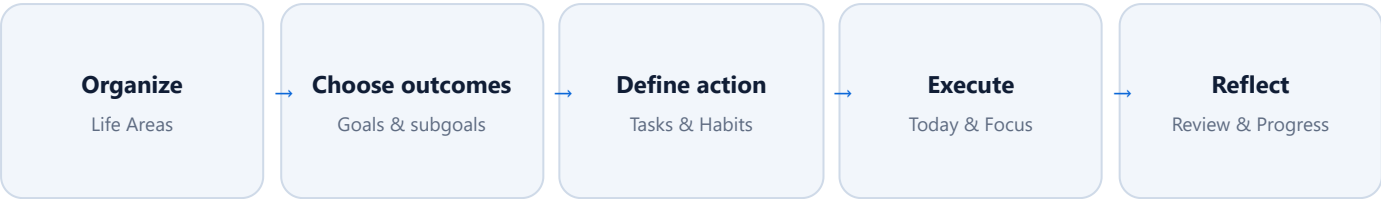
- 180 unit cases in the working tree.
- 32 Playwright tests plus auth setup.
- Persistence, stale-session, task-goal, Brain conversion, Task Map, keyboard and responsive coverage.

Immediate decision points

1. **Do not deploy current Task Map code until migration 0010 and its snapshot are tracked and deployment state is verified.**
2. **Do not run E2E or test-account commands against any valuable database.** Add an enforced disposable-database marker before the next run.
3. Treat current Settings JSON export as a convenience copy, not a complete backup or restore mechanism.
4. Fix Review state synchronization, habit outcome normalization, Calendar fetching/timezone, and Brain Dump pending-state recovery before relying on historical analytics.
5. Establish a production baseline: pinned runtime, CI gates, migration sequence, backup/restore drill, security headers, health/telemetry, and incident/recovery notes.

Product model and lifecycle

GoHa is a private single-owner ‘Life Operating System’ organized around translating intent into action and reflection.



Supporting surfaces

SURFACE	ROLE IN THE LOOP	CANONICAL IMPACT
Brain Dump	Fast inbox before structure	Converts atomically into Task, Goal, or Habit
Task Maps	Visual dependency/context planning	Links to canonical tasks and reflects their status
Calendar	Unified time view	Reads tasks, habit occurrences, and focus totals
Settings	Personal rules and recovery	Changes timezone/week/theme; restores archives; exports JSON
Command palette	Global navigation and search	Finds tasks/goals/areas, but routes only to modules

Explicit product boundary

The inspected implementation is intentionally single-owner and personal. It does not implement teams, social sharing, billing, AI generation, autonomous automation infrastructure, calendar synchronization, push/email/Telegram notifications, public domain APIs, or account self-deletion. Daily planning/reflection times are stored intentions only. The repository’s older design/reference material includes Life Score and other concepts that are not present as working calculations.

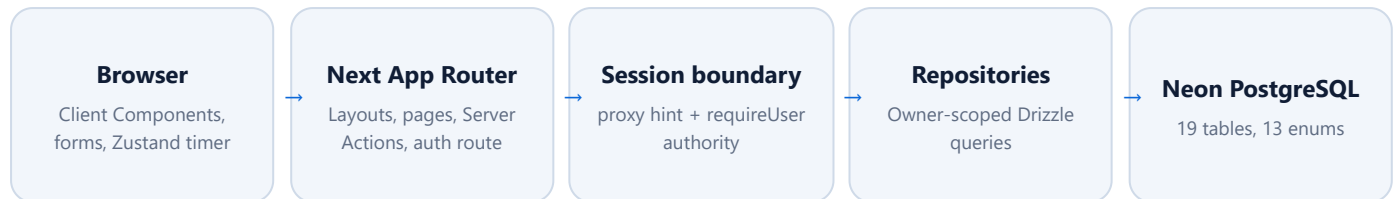
Single-owner does not mean unscoped data

Every domain row still carries `userId` and actions generally scope by the session owner. This is good future-proofing and defense in depth. The database, however, does not itself enforce cross-table owner equality or Row Level Security.

05

Architecture and request flow

The design is predominantly server-first: authenticated Server Components read repositories; Client Components manage interaction; Server Actions validate and mutate.



Read path

1. A protected request reaches `app/(app)/layout.tsx`. `proxy.ts` may first redirect based on session-cookie presence, but it does not establish identity.
2. `requireUser()` reads Better Auth session data from request cookies. React cache deduplicates session/settings reads within the request.
3. The async route page queries feature repositories, often in `Promise.all`. Identity is never accepted from the client.
4. Pure helpers derive view models: effective dates, streaks, progress, review totals, recommendations, calendar cells, and chart series.
5. The page sends serializable data to an interactive Client Component. There is no persistent repository cache or public data API.

Write path

1. A client form or interaction invokes a route-local Server Action.
2. The action calls `requireUser`, parses Zod input, and validates referenced record ownership where applicable.
3. A repository issues owner-scoped inserts/updates/deletes through the server-only Drizzle client.
4. The action calls `revalidatePath` for selected routes and returns a small success/error/result object.
5. Client state uses transitions, optimistic rendering, toasts, and celebrations to reconcile the response.

Boundary observations

- The only Route Handler is Better Auth's GET/POST catch-all; there is no public domain REST/GraphQL API.
- `db/index.ts` exports both repository namespaces and the raw db handle. Current feature code mostly respects repositories, but the raw escape hatch can bypass invariants.
- Neon HTTP is stateless and Better Auth/Drizzle interactive transactions are disabled. Multi-statement application workflows therefore require special care.
- React cache is request-local only. No use cache, unstable_cache, cache tags, service worker, or offline layer was found.

Technology and repository state

Versions, counts, reproducibility, and the exact condition of the audited working tree.

Technology stack

LAYER	INSTALLED VERSION / CHOICE	RESPONSIBILITY
Framework	Next.js 16.2.10 / App Router	Server Components, Server Actions, route boundaries, proxy
UI runtime	React 19.2.4	Client interactivity and transitions
Language	TypeScript 5.9.3, strict	Application, tests, scripts, and configuration
Styling	Tailwind CSS 4.3.2 + global CSS tokens	Responsive light/dark Apple-HIG-inspired interface
Motion	Motion 12.42.2	Spring transitions and reduced-motion-aware animation
Icons	Lucide React 1.23	Product and navigation iconography
Graph editor	@xyflow/react 12.11.2	Task Maps canvas, nodes, edges, minimap, viewport
Authentication	Better Auth 1.6.23	Email/password, sessions, account records, Next cookies
Data access	Drizzle ORM 0.45.2	Typed schema, repositories, migration generation
Database driver	@neondatabase/serverless 1.1	Stateless HTTP access to Neon PostgreSQL
Validation	Zod 4.4.3	Server Action and form input boundaries
Local state	Zustand 5.0.14	Only the active Focus timer store
Theme	next-themes 0.4.6	Immediate per-device light/dark/system state
Unit testing	Vitest 4.0.18 + jsdom	Pure domain/date/validation tests
Browser testing	Playwright 1.61.1	Chromium E2E, QA, auth state, screenshots
Package manager	pnpm lockfile v9	Resolved dependency graph and allowlisted install scripts

Source and repository counts

MEASURE	COUNT / STATE	INTERPRETATION
Tracked files	358	Approximately 10.77 MB; includes design references, PDFs, and generated Graphify output
Baseline working tree	31 modified + 6 untracked	Pre-report state; all changes belong to the user and were preserved
Core application TS/TSX/MTS	196	Across app, components, lib, db, stores; includes 113 TSX files
Server Actions	62	Exported async actions discovered under app/**/actions.ts
Pages	15	Root, login, register, and 12 authenticated feature pages
Route loading/error pairs	11	All primary features except Progress
Database	19 tables · 13 enums · 11 migrations	Four auth tables plus 15 domain tables
Unit tests	15 files · 180 cases	Two files / 30 cases are untracked
Playwright	8 specs · 32 tests + auth setup	Chromium only; 33 historical total including setup
Git history	16 commits · local main	No configured remote, tags, or releases

Snapshot reproducibility warning

The audited filesystem is newer than committed HEAD. Migration 0010, its snapshot, Task Map layout code, and two test files are untracked while related tracked schema/UI/journal files are modified. A clone of HEAD does not recreate the inspected system.

Package reproducibility

- Core Next/React versions are exact-pinned and pnpm-lock.yaml contains resolved integrity data.
- package.json lacks packageManager and engines. The current machine uses Node 24.12 and pnpm 11.10, while Next supports Node ≥ 20.9 .
- Project .mts commands rely on native/experimental TypeScript stripping that is not available across the full Next-supported Node 20 range. Pin a sufficiently new Node version or use a declared TS runner.
- pnpm-workspace.yaml explicitly allowlists required install build scripts, a useful supply-chain control.

Package and command surface

package.json declares 17 runtime dependencies, 18 development dependencies, and 16 scripts. It is private at version 0.1.0 but has no description, repository, license, or author metadata.

GROUP	SCRIPTS
Runtime	dev · build · start
Quality	lint · typecheck · test · test:watch · test:e2e
Database	db:generate · db:migrate · db:diagnose · db:push · db:studio
E2E account	test:account:create · test:account:reset · test:account:destroy

There is no combined check/CI command, formatter, production-mode E2E command, migration verification, or bundle-analysis command. CLAUDE.md names date-fns as mandatory, but the package and implementation use extensive custom date helpers instead. vite-tsconfig-paths and Testing Library packages are unused; the former brings deprecated unmaintained tsconfck, while deprecated esbuild-kit packages also arrive transitively through Drizzle Kit.

Route and shell catalog

Every reachable application surface and the shared desktop/mobile interaction frame.

Route matrix

ROUTE	ACCESS	PURPOSE	ENTRY FILE
/	Public redirect	Redirects to Today.	app/page.tsx
/login	Public auth	Email/password sign-in; accepts an intended local return path.	app/(auth)/login/page.tsx
/register	One-time bootstrap	Creates the first owner only; redirects after any user exists.	app/(auth)/register/page.tsx
/api/auth/[...all]	Route Handler	Better Auth GET/POST catch-all. The only Route Handler in the app.	app/api/auth/[...all]/route.ts
/today	Protected feature	Daily command center, recommendation, Top 3, tasks, habits, goals, momentum.	app/(app)/today/page.tsx
/goals	Protected feature	Goal hierarchy, progress, linked work, archives.	app/(app)/goals/page.tsx
/tasks	Protected feature	Task planning, detail, subtasks, completion reflection, task calendar.	app/(app)/tasks/page.tsx
/calendar	Protected feature	Read-only monthly task/habit/focus aggregation.	app/(app)/calendar/page.tsx
/habits	Protected feature	Boolean/numeric habit logging, schedules, streaks, weekly grid.	app/(app)/habits/page.tsx
/focus	Protected feature	Timestamp-driven focus timer and historical statistics.	app/(app)/focus/page.tsx
/brain-dump	Protected feature	Fast capture, sticky-note wall, conversion to canonical entities.	app/(app)/brain-dump/page.tsx
/task-maps	Protected feature	React Flow dependency maps linked to canonical tasks.	app/(app)/task-maps/page.tsx
/review	Protected feature	Derived weekly activity plus stored reflection and rating.	app/(app)/review/page.tsx
/progress	Protected feature	Twelve-week output/focus charts and 26-week habit heatmap.	app/(app)/progress/page.tsx
/life-areas	Protected feature	Life-area metadata and rollups.	app/(app)/life-areas/page.tsx
/settings	Protected feature	Profile, appearance, calendar rules, rhythm, password, archive, export.	app/(app)/settings/page.tsx

Protected shell

Desktop

- Fixed 260px sidebar with all 12 modules, owner identity, and logout.
- Sticky header with global search, Focus Mode, Add Task, and theme.
- Wide responsive content area without a hard reading-width ceiling.

Mobile

- Sticky glass header, hamburger drawer, brand, and theme.
- Bottom tabs: Today, To-dos, Habits, Settings; raised center Add Task.
- Drawer exposes the complete route set and locks body scrolling.

Command palette

- Cmd/Ctrl+K, navigation to all modules, New Task, Focus, Brain Dump, theme actions, lazy task/goal/life-area search, fuzzy subsequence matching, groups, arrows/Enter/Escape, focus restoration, and listbox semantics.
- Search is capped at 24 results and loaded once for the mounted shell. It becomes stale after record mutations until reload.
- Record results navigate only to module roots; the system has no entity-detail URL model. Palette theme changes local next-themes state but not the saved database preference.

Route boundaries

Today, Goals, Tasks, Calendar, Habits, Focus, Brain Dump, Task Maps, Review, Life Areas, and Settings each provide content-shaped loading.tsx and retryable error.tsx. Progress has neither. No custom app/global-error.tsx or app/not-found.tsx exists; framework defaults handle those cases.

Shell accessibility gaps

- The mobile drawer moves focus to Close, handles Escape, and locks scroll, but does not trap focus or restore it to the opener.
- Global toasts can overlap page content in the top-right.
- The palette is comparatively strong, but selected records cannot be deep-opened.

08

Feature-by-feature reference

Implementation, business rules, integration, and limitations for all twelve primary modules.

[/today](#)

Today — the daily command center

Today converts the entire system into a small executable surface: one explainable recommendation, up to three pinned priorities, current tasks and habits, active goals, and short-horizon momentum.

Primary implementation

- `app/(app)/today/page.tsx`
- `app/(app)/today/actions.ts`
- `components/today/**`
- `lib/today.ts`
- `lib/today-brain.ts`
- `lib/milestones.ts`

What the system does

- Loads top-level tasks and subtasks, goals with task counts, priority slots, habits plus roughly 400 days of entries, life areas, 14-day focus history, timezone, and week-start preference.
- Builds a local-date greeting, an explainable 'Today's Focus' card, Top 3 Actions, current/overdue task groups, Today habit controls, active goals, a momentum card, and quick task capture.
- 'Plan My Day' fills only empty priority slots and never replaces already pinned work. Top 3 uses canonical task records, not a second task model.
- Quick Add creates a canonical task scheduled for today. Task completion supports undo and can trigger a goal-progress celebration.
- DateRolloverRefresh refreshes the page when the owner's local date changes at midnight.

Rules and derived behavior

- Pinned open work wins. Otherwise scoring starts with lateness (100 plus four per late day, age contribution capped), then priority, in-progress state, due-today state, active-goal linkage, and an undated penalty.
- The recommendation exposes its reason; it is not an opaque AI decision. Derived values are recomputed from canonical task, goal, habit, and focus rows.
- Focus totals credit actual seconds capped to planned duration; Today uses a rolling seven-day focus sum and task completions by owner timezone.

Cross-feature integration

- Recommended work can open the shared Task Detail Panel or start Focus with the task preselected.
- Momentum links to Progress; active goals and habits use the same underlying records as their full modules.

Limitations and audit findings

- Several helper calls still default to Manila when deriving task effective dates, so a non-Manila owner can see edge-case inconsistencies despite Today itself computing its main date in the saved timezone.
- Top-right toast placement can cover the done-today summary. Tertiary copy is very faint in the supplied dark-mode screenshot.
- Automatic local-midnight refresh is not consistently present on Calendar, Progress, or Review.

[/tasks](#)

To-dos — canonical executable work

Tasks are the system's primary units of execution. They can stand alone or connect to a goal, life area, parent task, daily priority, focus session, or Task Map node.

Primary implementation

- `app/(app)/tasks/page.tsx`
- `app/(app)/tasks/actions.ts`
- `components/tasks/**`
- `lib/task-buckets.ts`
- `lib/tasks.ts`
- `lib/validations/task.ts`

What the system does

- Supports list and task-calendar layouts; independent timeframe filters (All, Today, Week, Month, Quarter, Year, Inbox); status filters; life-area filters; and date, priority, newest, or title sorting.
- Creation/editing covers title, description, status, priority, goal, life area, scheduled date, and timezone-interpreted due date/time. Query parameters can open New Task and preserve a goal selection.
- The detail panel autosaves editable properties, manages one level of subtasks, stores a completion reflection, and separates cancellation from confirmed hard deletion.
- The task calendar uses scheduled date before due-date-local-day, creates a predated task from a day, and keeps undated work in an Inbox section.

Rules and derived behavior

- Completion sets `completedAt` and optional `completionNote`; reopen and cancel clear `completedAt`. Goal auto-progress excludes cancelled tasks.
- One-level subtask depth is enforced by actions, not by a database constraint. Parent task deletion cascades its subtasks.
- The database has `estimateMinutes`, but current form validation and UI do not expose or consume it.

Cross-feature integration

- Completion affects Today, Goal auto-progress, Progress, Review, Calendar, and linked Task Map status.
- Focus sessions may link to a task; deleting the task sets those links and Task Map links to null.

Limitations and audit findings

- There is no task-detail URL. Search, Calendar, Review, Brain conversion, and Task Maps can navigate only to the general Tasks route.
- List cards open the detail panel while calendar entries open the edit modal, creating two primary interactions for the same entity.
- Repeated completion restamps `completedAt`; completing without a note can clear an existing completion note.

[/goals](#)

Goals — outcomes and progress

Goals connect strategic outcomes to life areas, subgoals, and tasks. Progress can be derived automatically or maintained manually with a history journal.

Primary implementation

- `app/(app)/goals/page.tsx`
- `app/(app)/goals/actions.ts`
- `components/goals/**`
- `lib/goals.ts`
- `lib/goal-progress.ts`
- `lib/validations/goal.ts`

What the system does

- Responsive card grid with timeframe tabs, five lifecycle states, start/target dates, priority, life-area linkage, and parent/subgoal hierarchy.
- Automatic progress is completed direct tasks divided by non-cancelled direct tasks; a completed goal is always displayed as 100%. Manual mode accepts an integer 0–100 and can append a progress note.
- Detail displays lineage, progress source, direct subgoals, and linked tasks; Add Task opens task creation with the goal preselected. Archive is recoverable from Settings.

Rules and derived behavior

- The parent chooser excludes the goal and its descendants. Direct self-parenting is also blocked in SQL, but longer cycles remain an application invariant.
- Completing the last task can produce a celebration but does not automatically change the goal's status to completed.
- Manual progress history is append-only by convention, not immutable at the database layer.

Cross-feature integration

- Goal linkage informs Today's recommendation, Task filters, Focus aggregation, Task Maps, Review, Progress, and Life Area rollups.

Limitations and audit findings

- Only direct subgoals are surfaced; there is no full goal tree.
- Parent auto-progress does not roll up child-goal progress.
- Goal update and progress-journal insertion are separate database statements, so partial success is possible.
- Start date may be after target date; crafted calls can reference archived records.

[/calendar](#)

Calendar — unified read-only schedule

Calendar presents tasks, habit occurrences, and completed focus totals together in a monthly planning surface without duplicating their source records.

Primary implementation

- `app/(app)/calendar/page.tsx`
- `components/calendar/calendar-view.tsx`
- `lib/calendar.ts`
- `lib/task-buckets.ts`

What the system does

- Month grid with configurable week start, previous/Today/next navigation, layer selector, life-area task colors, and a selected-day detail card.
- Tasks are loaded broadly; habit entries and completed focus sessions are loaded into an initial date window. Future habit occurrences are described as scheduled rather than failed.

Rules and derived behavior

- A task uses scheduled date first, otherwise the timezone-local date of its due instant.
- Habit occurrences are generated from schedule rules; focus sessions are aggregated by sessionDate.

Cross-feature integration

- Reads canonical Tasks, Habits, Focus, Life Area colors, and settings; it does not mutate them.

Limitations and audit findings

- **Silent range truncation:** `app/(app)/calendar/page.tsx:19-33` fetches habit/focus only ± 62 days around the initial today, but the client permits unlimited month paging and does not refetch. Outside the window, those layers silently disappear.
- **Timezone defect:** the page reads the saved timezone but does not pass it to `CalendarView`; `components/calendar/calendar-view.tsx` calls `taskEffectiveDate` with the Manila default.
- The grid obeys layer filters, but the selected-day detail remains unfiltered. The Today button changes the month anchor without resetting the selected day.
- Raw habit status treats any numeric entry stored as done as complete, even when it misses its target. Monthly/unspecified-weekday schedules are rendered as daily occurrences.
- Selected-day records are plain text; no entity can be opened directly.

[/habits](#)

Habits — repeatable behavior and streaks

Habits track boolean or numeric behavior with schedule-aware outcomes, weekly presentation, and current/longest streaks.

Primary implementation

- `app/(app)/habits/page.tsx`
- `app/(app)/habits/actions.ts`
- `components/habits/**`
- `lib/habits.ts`
- `lib/habit-view.ts`
- `lib/habit-streaks.ts`
- `lib/validations/habit.ts`

What the system does

- Boolean and numeric types; targets and units; higher-is-better or lower-is-better; life-area/goal linkage; color/icon; active schedule; archive/restore.
- UI exposes daily, chosen weekdays, and X-times-per-week. Logging supports done, missed, skipped, clear, and numeric value entry with optional note at the data layer.
- Weekly grid distinguishes done, partial, missed, skipped, pending, and off-schedule. Milestones can celebrate 7-day and 30-day streaks.

Rules and derived behavior

- Only scheduled days break day-based streaks; today stays pending; skips are neutral; below-target numeric entries break the streak.
- X-times-per-week habits use consecutive successful weeks, but are still shown as eligible every day by generic occurrence logic.
- The schema supports monthly schedules, `daysOfMonth`, and `endDate`, but the UI does not expose monthly scheduling and core eligibility ignores `daysOfMonth/endDate`.

Cross-feature integration

- Habit outcomes feed Today, Calendar, Progress heatmap, Weekly Review, and Life Area rollups.

Limitations and audit findings

- **Cross-screen contradiction:** logging a numeric value stores raw status `done` even below target. Habits/Today correct this with `entryOutcome`; Calendar and wide-window Progress/Review paths read the raw status and count it complete.
- Schedule editing updates the current active row in place despite schema comments describing history. No database rule guarantees one active schedule per habit.
- Times-per-period, weekday/month-day ranges, schedule start/end ordering, habit-type/value consistency, and future/ancient entry dates are insufficiently constrained.
- Create/update plus schedule write are non-atomic.

[/focus](#)

Focus — durable timestamp-based work sessions

Focus links concentrated time to an optional task and derives reliable elapsed/credited time from server timestamps rather than a fragile interval counter.

Primary implementation

- `app/(app)/focus/page.tsx`
- `app/(app)/focus/actions.ts`
- `components/focus/**`
- `stores/focus-timer.ts`
- `lib/focus.ts`
- `lib/focus-reconcile.ts`

What the system does

- Preset durations 15/25/45/60, custom 1–240 minutes, optional open task, pause/resume, extend by preset or custom 1–120, complete, abandon/discard, completion note, recent sessions, and task/goal/life-area statistics.
- Timer survives sleep and reload because it derives remaining time from timestamps and accumulated pauses. It may overrun at zero until the user acts.
- Sessions older than 12 hours are reconciled when Focus loads; credited time is capped to planned duration, with a maximum four hours for stale reconciliation.

Rules and derived behavior

- The database is authoritative; Zustand supplies only live interaction state.
- Starting a session first abandons all detected open sessions, then creates the new one.
- `sessionDate` is currently set with `manilaToday` in the repository, ignoring the saved timezone.

Cross-feature integration

- Today can preselect the recommended task. Completed sessions feed Today momentum, Calendar, Review, and Progress.

Limitations and audit findings

- No notification, sound, or automatic completion; sessions may remain at zero indefinitely.
- Notes exist only in local component state until finish and are lost on refresh/navigation. Discard is immediate and unconfirmed.
- Invalid custom text leaves the previous valid duration in parent state, so Start can remain possible.
- No partial unique index enforces one active session. Close-then-create is non-atomic; concurrent starts can produce multiple active rows.

[/brain-dump](#)

Brain Dump — capture and canonical conversion

Brain Dump is the low-friction inbox. It keeps loose thoughts editable and can atomically transform one into a canonical task, goal, or habit.

Primary implementation

- `app/(app)/brain-dump/page.tsx`
- `app/(app)/brain-dump/actions.ts`
- `components/brain-dump/**`
- `db/repositories/brain-dump.ts`
- `lib/brain-dump.ts`
- `lib/validations/brain-dump.ts`

What the system does

- Fast capture up to 2,000 characters, Cmd/Ctrl+Enter, six colors, optimistic sticky-note wall, Inbox/Archived/Converted filters, edit/recolor/archive/restore/confirmed delete.
- Conversion uses a writable CTE and FOR UPDATE lock, creates the target row, marks the note converted, and stores target type/id in one parameterized database operation.

Rules and derived behavior

- Converted items cannot be edited or converted twice.
- Task conversion creates a basic task; goal conversion a monthly goal; habit conversion a boolean daily habit plus schedule.

Cross-feature integration

- Conversion targets the same Tasks, Goals, and Habits used everywhere else. Converted notes remain visible and offer a generic destination link.

Limitations and audit findings

- **Known race:** `docs/BUILD_PLAN.md:613–624` and `e2e/qa.spec.ts:400–415` document rapid repeated capture (roughly under 400 ms) leaving the optimistic transition permanently pending until reload.
- The 2,000-character source is copied whole into target name/title columns whose action edit limits are 160/120/80. PostgreSQL text has no equivalent length limit, so converted records can be impossible to edit without shortening.
- Conversion has no setup step for area, goal, priority, schedule, or habit type. View opens only the general module.
- A later hard-delete of the converted target leaves a dangling polymorphic ID. Create and recolor are separate actions; recolor failure is silently ignored.

[/task-maps](#)

Task Maps — dependency visualization

Task Maps is a React Flow workspace for planning dependencies and context around canonical tasks, with semantic free-form nodes and persisted geometry.

Primary implementation

- `app/(app)/task-maps/page.tsx`
- `app/(app)/task-maps/actions.ts`
- `components/task-maps/task-maps-workspace.tsx`
- `components/task-maps/flow-canvas.tsx`
- `lib/task-maps.ts`
- `lib/task-map-layout.ts`
- `lib/validations/task-maps.ts`

What the system does

- Create/rename/describe/archive/restore/delete maps; task, note, group, milestone, decision, blocker, and phase nodes; node note/type/color/task link; loose labeled edges; minimap; grid; zoom; fit; viewport persistence; legend; batch task import; and Tidy layout.
- Linked task nodes show canonical status/priority. Position saves are debounced 500 ms and viewport saves 600 ms. Duplicate directed edges are resolved idempotently.

Rules and derived behavior

- Map progress counts linked nodes, not distinct task IDs. Edge uniqueness is map/source/target and self-loops are blocked.
- Position 'batch' updates are Promise.all over individual writes, not one transaction.

Cross-feature integration

- Maps can link to a goal/life area and import canonical tasks. Task deletion nulls node task links; map deletion cascades nodes/edges.

Limitations and audit findings

- Archived maps remain fully editable.
- Node and edge deletes are immediate and optimistic, with no confirmation and no rollback after server failure.
- Inspector linkage permits the same task on multiple nodes; progress is inflated and cancelled tasks remain counted.
- Import Tasks does not use the shared accessible modal and lacks equivalent focus trapping/restoration/scroll lock.
- Tidy's cycle guard can still move reachable cyclic nodes to extreme layers; disconnected nodes can form a very wide row. The supplied QA screenshot shows tiny unreadable fitted nodes and inspector occlusion.
- Migration 0010 required for three node types and notes is untracked in the audited baseline, creating deployment drift.

[/review](#)

Weekly Review — reflection over live canonical history

Review combines derived weekly activity with saved prose and a rating. It supports drafts, completion/locking, reopening, and past-week navigation.

Primary implementation

- `app/(app)/review/page.tsx`
- `app/(app)/review/actions.ts`
- `components/review/review-view.tsx`
- `lib/review.ts`
- `db/repositories/reviews.ts`

What the system does

- Week selected by query string, malformed/future dates clamped to the current week, unlimited prior navigation, next disabled at current week.
- Derives completed tasks by completedAt, slipped/open work, life-area effort, focus time/sessions, habit completion, and goals completed; stores wins, challenges, next-week focus, optional 1–5 rating, and completion timestamp.

Rules and derived behavior

- Weekly statistics remain live derivations; they are not snapshotted at completion. Later edits can change the visible history.
- Unique owner/week start makes upsert idempotent. `listWeeklyReviews` is named as if it lists finished reviews but does not filter drafts.

Cross-feature integration

- Reads Tasks, Habits, Focus, Goals, Life Areas, and Settings week start/timezone.

Limitations and audit findings

- **High-confidence state defect:** `components/review/review-view.tsx:84–87` initializes four `useState` values from props but does not key or synchronize them when `weekStart` changes. Client navigation can carry prior-week prose/rating into a new week and save it under the new key.
- No dirty-state warning or autosave; navigation can lose unsaved reflection.
- Carry-over is derived but not rendered. 'Focus next week' is prose only and creates no priority/task/goal.
- It inherits the wide-window numeric-habit completion contradiction. Crafted review actions can accept non-configured week-start dates.

[/progress](#)

Progress — historical output and consistency

Progress visualizes recent task completion, focus time, habit consistency, current streaks, and goal counts from canonical records.

Primary implementation

- `app/(app)/progress/page.tsx`
- `components/progress/progress-view.tsx`
- `components/charts/bar-series.tsx`
- `components/charts/habit-heatmap.tsx`
- `lib/progress.ts`

What the system does

- Twelve weekly task-completion bars, twelve focus-minute bars, 26-week habit heatmap, current summary cards with comparison deltas, best habit streak, active-goal count, and an activity-based empty state.
- Task attribution uses `completedAt` in saved timezone; focus summary preserves seconds even though bars display minutes.

Rules and derived behavior

- Current windows begin at the week containing eleven weeks ago and end today; heatmap begins at the week containing 25 weeks ago.
- Longest streak is limited to the entries fetched for the page window.

Cross-feature integration

- Reads Tasks, Habits, Focus, Goals, and date preferences. Today's Momentum card links [here](#).

Limitations and audit findings

- Current task/focus range is partial through today while the prior comparison is twelve full weeks, producing unequal-window deltas.
- Habit Rate displays a 26-week measure while its previous baseline comes from a separate 12-week range.
- Wide-window numeric entries can be counted complete from raw status; an older lifetime-best streak can disappear outside the fetched window.
- `dailyFocusMinutes` and `goalsCompleted` are prepared but not rendered; goals alone do not satisfy `hasAnyHistory`.
- Charts are `div/span` graphics with mouse title text, no keyboard-accessible cells or equivalent data table.
- This is the only primary route without `loading.tsx` and `error.tsx`.

[/life-areas](#)

Life Areas — organizing domains

Life Areas categorize goals, tasks, and habits and provide stable color/icon identity plus basic rollups.

Primary implementation

- `app/(app)/life-areas/page.tsx`
- `app/(app)/life-areas/actions.ts`
- `components/life-areas/**`
- `lib/life-areas.ts`
- `lib/validations/life-area.ts`

What the system does

- Create/edit/archive/restore; name, description, color, icon, importance 1–5, and sort order.
- Cards show active goals, open/completed tasks, active habits, and task completion. Colors propagate to Goals, Tasks, Calendar, Review, and related chips.

Rules and derived behavior

- Archiving a life area does not archive or unlink child goals/habits/tasks. Missing explicit color falls back to a stable ID hash.
- Deletion is not exposed; archive recovery is centralized in Settings.

Cross-feature integration

- Provides taxonomy and visual identity across most feature modules.

Limitations and audit findings

- Cards do not drill into filtered work.
- Importance copy says it weights life balance, but no inspected calculation consumes the weight.
- The product/design concept of a Life Score or area balance trend is not implemented.

[/settings](#)

Settings — identity, preferences, recovery, export

Settings controls the owner profile, visual and calendar preferences, daily intentions, password, archives, and local JSON export.

Primary implementation

- `app/(app)/settings/page.tsx`
- `app/(app)/settings/actions.ts`
- `app/(app)/settings/archive-actions.ts`
- `app/(app)/settings/export-actions.ts`
- `components/settings/settings-view.tsx`
- `lib/user-settings.ts`
- `lib/timezones.ts`
- `lib/validations/settings.ts`

What the system does

- Display name; read-only email; light/dark/system; curated IANA timezone with fallback; Monday/Sunday/Saturday week start; planning/reflection times; password change with current-password verification and revocation of other sessions.
- Central archive lists/restores life areas, goals, habits, and maps. JSON export downloads plaintext data in the browser and intentionally excludes password hashes, session tokens, and provider tokens.

Rules and derived behavior

- Theme is split between database preference and next-themes/local storage. ThemeSync seeds DB choice only when the device lacks a local choice.
- Rhythm times are intentions only: no notification or automation is implemented.

Cross-feature integration

- Timezone and week start affect Today, task dates, habit weeks, Calendar, Review, and Progress after refresh.

Limitations and audit findings

- The command palette changes local theme without persisting DB, while header and Settings persist both.
- Several preference controls optimistically update local state and do not roll back on persistence failure.
- No password confirmation/show-hide, recovery, session browser, email change, data import, or account deletion.
- Export is materially incomplete; the dedicated export chapter enumerates every omission.

Cross-feature consistency

The greatest strength of GoHa is shared canonical data. The most important defects are places where different readers interpret the same row differently.

CANONICAL EVENT	EXPECTED DOWNSTREAM READERS	OBSERVED CAVEAT
Task create/edit	Tasks · Today · Calendar · Goal · Task Map search · command search	Explicit revalidation omits some dependent paths; command index stays stale until reload
Task completion	Today · Goal auto-progress · Progress · Review · Calendar · Task Map	Repeated completion restamps completedAt; goal status is not automatically completed
Habit log	Habits · Today · Calendar · Progress · Review	Below-target numeric values disagree because readers mix derived outcome and raw done status
Focus completion	Focus · Today · Calendar · Progress · Review	sessionDate is Manila-derived; starting/replacing session is non-atomic
Life Area change	Goals · Tasks · Habits · Calendar · Review · Focus rollups	Archived references remain possible through crafted calls; no direct area drill-down
Brain conversion	Task/Goal/Habit modules · Converted inbox	Target title length can exceed later edit validation; View is not a deep link
Archive	Normal modules hide · Settings restores	Archived maps remain editable; archive action does not cascade to child entities
Timezone/week setting	Today · Tasks · Habits · Calendar · Review · Progress · Focus · Export	Several callers still use Asia/Manila defaults

Revalidation map

- Task actions refresh Tasks, Goals, and Today.
- Habit actions refresh Habits and Today.
- Focus actions refresh Focus.
- Life Area actions refresh Life Areas; Goal actions refresh Goals; map actions normally refresh Task Maps.
- Brain conversion refreshes Brain Dump, the target module, and Today. Preferences refresh the root layout. Review refreshes Review.
- Calendar, Progress, Review, command search, and Life Area rollups are often absent from explicit dependency revalidation. Direct uncached reads are fresh on a full request, but client Router Cache/local state behavior becomes fragile.

Canonical normalization recommendation

Create one shared habit-outcome function at the repository/view-model boundary and require Calendar, Review, Progress, Today, and Habits to consume it. Likewise, create one required date-context object rather than optional timezone defaults for user-facing bucketing.

10

Data model: 19 tables

Four Better Auth tables and fifteen user-owned domain tables form a connected PostgreSQL model. UUID primary keys and TIMESTAMPTZ audit fields are the norm.

Complete table catalog

TABLE	GROUP	PRINCIPAL FIELDS	CONSTRAINTS AND DELETION BEHAVIOR
<code>user</code>	Better Auth	id, name, unique email, verified flag, image, timestamps	Owner root; deleting user cascades most linked rows.
<code>session</code>	Better Auth	unique token, expiry, IP, user agent, owner, timestamps	User delete cascades. Session cookie is server-validated.
<code>account</code>	Better Auth	provider/account IDs, tokens, expiries, scope, password hash, owner	User delete cascades; no unique (providerId, accountId).
<code>verification</code>	Better Auth	identifier, value, expiry, timestamps	No user FK; verification rows do not cascade with user.
<code>life_areas</code>	Domain	owner, name/description, color/icon, positive weight, order, archive	Owner indexes; children link with SET NULL rather than archive cascade.
<code>goals</code>	Domain	owner, life area, parent, title/description, status/timeframe/priority, progress mode/value, dates, archive	Manual progress 0–100; no direct self-parent; parent deletion cascades child goals.
<code>goal_progress_updates</code>	Domain	owner, goal, progress 0–100, note, createdAt	Goal delete cascades. Append-only is a repository convention.
<code>tasks</code>	Domain	owner, goal/area/parent, title/description, status/priority, scheduled date, due instant, completion fields, estimate	Parent delete cascades subtasks; goal/area deletion SET NULL; estimate positive when present.
<code>habits</code>	Domain	owner, goal/area, name/description, type, target/unit/direction, color/icon, archive	Numeric habits require target; goal/area deletion SET NULL.
<code>habit_schedules</code>	Domain	owner, habit, frequency, weekday/month-day arrays, times/period, start/end, active	Habit delete cascades; no one-active-schedule uniqueness.
<code>habit_entries</code>	Domain	owner, habit, local date, status, value, note	Unique habit/date; non-negative value; habit delete cascades.
<code>focus_sessions</code>	Domain	owner, optional task, sessionDate, timestamps, pauses, planned/actual duration, status, note	Task deletion SET NULL; no one-active-session or temporal consistency constraint.
<code>brain_dump_items</code>	Domain	owner, content, status/color, converted type/id, order	Polymorphic target is not an FK; both conversion columns null or both set.
<code>daily_priorities</code>	Domain	owner, local date, position 1–3, optional task/label, label-completed	Unique owner/date/position; task delete cascades; does not prevent duplicate task slots.

TABLE	GROUP	PRINCIPAL FIELDS	CONSTRAINTS AND DELETION BEHAVIOR
task_maps	Domain	owner, name/description, optional goal/area, JSON viewport/legend, archive	Owner scoped; map delete cascades graph.
task_map_nodes	Domain	owner, map, optional task, seven node types, label/note, position/size/data	Map delete cascades; task deletion SET NULL; same-owner not enforced by FK.
task_map_edges	Domain	owner, map, source/target, label/data	Unique map/source/target; no self-loop; endpoint/map consistency not enforced.
weekly_reviews	Domain	owner, week start, three prose fields, rating, completedAt, timestamps	Unique owner/week; rating 1–5 when present.
user_settings	Domain	unique owner, timezone, theme, week start, rhythm, notification preference, onboarding, JSON preferences	Week start 0–6; default timezone Asia/Manila.

PostgreSQL enums

ENUM	VALUES
goal_status	not_started · active · paused · completed · dropped
goal_timeframe	daily · weekly · monthly · quarterly · yearly
goal_progress_mode	auto · manual
task_status	todo · in_progress · completed · cancelled
priority	low · medium · high · urgent
habit_type	boolean · numeric
habit_frequency	daily · weekly · monthly
habit_entry_status	done · missed · skipped
focus_status	in_progress · completed · abandoned
brain_dump_status	inbox · converted · archived
brain_dump_converted_type	task · goal · habit
theme	light · dark · system
task_map_node_type	task · note · group · milestone · decision · blocker · phase

Ownership model

- Every protected action derives identity from session cookies through requireUser; no domain action accepts an arbitrary userId from the browser.
- Update/delete queries generally match both record ID and owner. Referenced goal/area/task/map ownership is explicitly checked in current actions.
- Aggregate subqueries in Life Areas and Goals also independently filter the owner, which prevents cross-owner count leakage in normal operation.

- The database does not use RLS or composite (userId,id) foreign keys. Child userId and parent userId can disagree if a script or future bug bypasses action checks.
- Task Map graph reads verify map ownership and then select nodes/edges by taskMapId only; inconsistent pre-existing cross-owner rows would be returned.

Database constraints that are missing

- Exactly one owner account; exactly one active focus session per user; exactly one active schedule per habit.
- Cross-table owner equality for goal/area/task/habit/schedule/entry/progress/map/node/edge relationships.
- Focus temporal/status consistency and positive planned duration.
- Goal start $\leq$ target, hierarchy acyclicity beyond direct self-parenting, one-level task hierarchy.
- Schedule weekday/month-day ranges, positive timesPerPeriod, start $\leq$ end, and endDate enforcement in product logic.
- Habit entry type/value consistency; daily-priority XOR and task uniqueness across three slots.
- Task Map edge endpoints belonging to the same map and owner; polymorphic Brain conversion target existence and consistency between status=converted and conversion metadata.
- Database-level text limits matching Zod action limits.

The explicit weekly review owner/week index duplicates the same leading structure already supplied by its unique constraint. This is harmless functionally but redundant maintenance/storage.

Update timestamps

Most mutable tables use Drizzle-level `$onUpdate` behavior from `db/schema/_shared.ts`, not a PostgreSQL trigger. Writes that bypass Drizzle do not automatically receive the same `updatedAt` semantics.

Migrations and schema evolution

The repository contains a sequential 0000–0010 history, but two legacy transitions lack data backfills and the newest migration is not tracked.

NO.	MIGRATION	CHANGE
0000	init_full_schema	Initial auth and domain schema: original task done status, priority through high, four map node types, boolean habit completed, task-only Brain conversion.
0001	tasks_status_priority_feedback	Renames task done→completed, adds urgent, adds completion_note.
0002	habit_entry_status_add	Adds done/missed/skipped status with done default.
0003	habit_entry_drop_completed	Drops old habit completed boolean.
0004	focus_pause_tracking	Adds pause timestamp, accumulated paused seconds, index, and check.
0005	brain_dump_converted_add	Adds task/goal/habit conversion enum plus polymorphic type/id.
0006	brain_dump_drop_task_id	Drops legacy converted_task_id FK/index/column.
0007	busy_whizzer	Adds Brain note color.
0008	slow_silhouette	Adds task parent/subtasks and Task Map legend.
0009	shiny_vengeance	Adds weekly_reviews.
0010	jazzy_husk	Adds decision/blocker/phase map-node enum values and task_map_nodes.note. SQL and snapshot are untracked in the baseline.

Migration safety findings

0002 → 0003: habit meaning can be lost

0002 adds status with a default of done to every existing entry. It does not map old completed=false rows to missed or another neutral state. 0003 then drops completed. A populated deployment would misclassify all prior false rows as done.

0005 → 0006: conversion provenance can be lost

0005 adds converted_type and converted_entity_id but does not populate them from converted_task_id. 0006 drops the legacy task link. Existing converted-task provenance can disappear.

0010: filesystem and commit history diverge

The SQL and snapshot exist locally but are untracked. The tracked journal/schema and application UI already depend on them, and BUILD_PLAN states they were applied. Deployment state cannot be proven without querying the database.

Operational migration gaps

- No CI migration-drift check, shadow/apply verification, or deployment ordering workflow.
- db:push can bypass migration history and has no environment guard.
- No rollback/down strategy, restore drill, or migration-before-code runbook.
- Runtime and migration tools share DATABASE_URL; no least-privilege separation is designed into the environment contract.

Authentication, authorization, and security

Current application-layer owner scoping is generally careful, but bootstrap, redirect, database invariants, and production hardening need attention.

Authentication behavior

- Email/password with auto sign-in; password length 8–128; UUID Better Auth IDs; nextCookies plugin; seven-day installed default session behavior.
- `user.create.before` asks `usersRepo.hasAnyUser` and rejects registration after the first owner. `/register` independently redirects after any account exists.
- proxy performs only an optimistic cookie-presence redirect. Protected layout and actions authoritatively validate the session. Stale cookies are allowed to reach Login to avoid a redirect loop.
- Password change verifies the current password and revokes other sessions. Email change, reset delivery, recovery, email verification requirement, and 2FA are not configured.
- Installed Better Auth production rate limiting defaults to process memory, so counters do not coordinate reliably across serverless instances.

Authorization strengths

- Session identity, not browser-supplied owner ID, drives every protected page/action.
- Most repositories filter ID and owner together; foreign/invalid records receive generic not-found behavior.
- Brain conversion locks and owner-filters its source row and uses parameterized SQL.
- Database modules are marked server-only; no `NEXT_PUBLIC` secret usage, `dangerouslySetInnerHTML`, `eval`, or equivalent injection sink was found.
- `.env.local` and saved Playwright auth are ignored; no tracked production-secret pattern was found.

Security findings

- First-owner creation is a read-then-create race with no singleton constraint or serializable lock. Concurrent distinct registrations can both pass.
- `scripts/test-account.mts` directly inserts an additional known-credential account and has no proof that `DATABASE_URL` is disposable; it also prints the account email inventory to logs.
- Login server and AuthForm client accept any redirect beginning with slash, including a protocol-relative `//host`. The login screen continues to advertise the owner-creation link after bootstrap, where it simply redirects back.
- No RLS/composite same-owner FKs; raw db handle remains exported.
- `next.config.ts` defines no CSP, HSTS, Referrer-Policy, Permissions-Policy, X-Content-Type-Options, frame policy, or `poweredByHeader:false`.
- No security audit log, centralized error sink, account recovery, 2FA, session browser, or account deletion.
- Action error logging may include raw driver errors. `scripts/diagnose.mts` prints email/name/UUID/task titles/Brain excerpts/settings plus database host/name/role and is unsafe for shared logs.

Privacy model

GoHa stores highly personal goals, reflections, habits, notes, schedules, session IP/user-agent information, and activity history. Protection relies on TLS/platform/database controls; no application-layer content encryption or documented retention policy exists. The JSON export is plaintext and contains email plus personal content without an encryption option or prominent warning.

Validation, transactions, concurrency, and caching

The action boundary is typed and owner-aware, but several invariants remain only conventions and multi-statement writes are not atomic.

Primary validation limits

ENTITY	LIMITS
Life Area	name 60 · description 280 · weight 1–5
Goal	title 120 · description 500 · manual progress integer 0–100
Task	title 160 · description 1000 · completion note 1000
Habit	name 80 · description 280 · unit 24 · entry note 280
Brain Dump	content 2000
Task Map	name 120 · description 500 · node label 200 · node note 2000 · edge/legend label 40
Focus	note 2000 · planned 1–240 min · extension 1–120 min/request
Review / profile	prose 2000 · rating 1–5 · profile name 80

Validation gaps

- Goal date ordering and manual-mode/value consistency; archived references in crafted calls.
- Numeric habit value requirement and boolean-value rejection; entry dates can be arbitrary future/ancient values.
- Review complete argument is not itself parsed; crafted dates may not match configured week start.
- Settings time regex validates shape, so 99:99 can reach PostgreSQL and fail generically.
- Date conversion does not round-trip calendar validity or DST gaps/ambiguity; Date.UTC normalization can accept crafted invalid wall dates.
- Task Map zoom/coordinates need only be finite; zero, negative, and extreme values are accepted.

Non-atomic workflows

WORKFLOW	STATEMENTS	POSSIBLE PARTIAL STATE
Habit create/update	Habit then schedule	Habit changed without schedule or schedule change
Manual goal progress	Goal then progress journal	Displayed value changes without history entry

WORKFLOW	STATEMENTS	POSSIBLE PARTIAL STATE
Focus start	Abandon open sessions then insert	Old session closed without replacement; concurrent multiple active rows
Plan My Day	Multiple slot upserts	Only some empty Top 3 positions filled
Task Map import/position	Many independent writes	Partial import/layout persistence
Better Auth/test account	Sequential auth rows	Partially created account/session structures

Caching and stale client state

- No persistent data cache; protected pages are dynamic and query Neon on a new request.
- React cache deduplicates session/settings inside one render only.
- Server Actions use selected path revalidation, but explicit dependency coverage is incomplete.
- Command search is cached for the mounted shell; Review, Focus notes, Settings optimistic choices, and palette theme demonstrate local state that can diverge from new server props or failed persistence.

Export, archive, deletion, and privacy lifecycle

Soft archive is thoughtfully integrated, but export is not a complete backup and several data classes have no recovery path.

Export completeness audit

INCLUDED	MISSING OR TRUNCATED
Account name/email	Auth sessions, password hashes, provider tokens intentionally excluded (correct)
Life areas and goals	Goal progress-update journal
Top-level tasks and subtasks	Daily priority slots
Habits with one attached active schedule	Inactive/historical schedules and any additional active rows
Habit entries in a ten-year past window through today	Older entries and future-dated entries
Recent finished/abandoned focus history	In-progress sessions and rows after a 10,000 cap
Brain Dump items	No target integrity verification for dangling conversions
Task Map records	All Task Map nodes and edges
Weekly reviews	Rows beyond a 1,000 cap
Settings	No import/restore path for any exported structure

Export wording is unsafe

The Server Action and historical BUILD_PLAN describe ‘everything’/every canonical row, but the implementation omits entire tables and imposes silent ranges/caps. It also uses Manila rather than the saved timezone for the window and filename. Treat it as a convenience export only.

Soft archive

- Life Areas, Goals, Habits, and Task Maps can be archived and restored from Settings.
- Archiving does not make a record immutable and does not cascade archive state to linked children.
- Archived Task Maps remain editable in the current workspace.
- Archive recovery is comprehensive but distant from the action that caused it.

Hard deletion and retention

- Tasks/subtasks, Brain items, daily priorities, habit entries, in-progress Focus sessions, map nodes/edges, and entire Task Maps can be hard-deleted.
- Finished Focus sessions and Weekly Reviews have no delete action. Goal progress history is append-only only by convention.
- No account-deletion UI, automated retention/pruning, recycle bin for hard-deleted rows, audit log, or import/restore.
- Deleting a target may leave Brain conversion provenance dangling. User deletion cascades almost all owned data; verification is not user-linked.

Design, responsive UI, and accessibility

The shipped interface intentionally supersedes the older Serene Lifecycle references with an Apple-HIG/Liquid-Glass direction.

Current visual language

- Cool gray light canvas and true-black dark canvas; iOS-style blue action color; solid content cards; glass for navigation, dialogs, headers, and floating controls.
- Inter for UI and Geist Mono for numerical text, CSS variable tokens, thin separators, concentric radii, tabular figures, spring motion, and separate dark palette.
- Reduced-motion and reduced-transparency/no-backdrop-filter fallbacks are present. Global .hit-44 supports 44px target sizing.
- Older Stitch exports use cream/sage, Manrope, a 280px sidebar, Material-like surfaces, and are historical design references rather than the current source of truth.

Shared primitive strengths

- Button retains accessible name in loading state and exposes aria-busy; Checkbox has enlarged target and native-like state semantics.
- Shared Modal and Detail Panel trap focus, handle Escape/backdrop, lock body scroll, restore focus, and become mobile bottom sheets.
- Segmented Control implements radiogroup arrows/Home/End; Dropdown implements menu navigation/typeahead; Select provides keyboard navigation and portal positioning; Progress exposes semantic values.
- Destructive whole-entity flows usually ask confirmation; focus rings are visible; Sonner announces toasts.

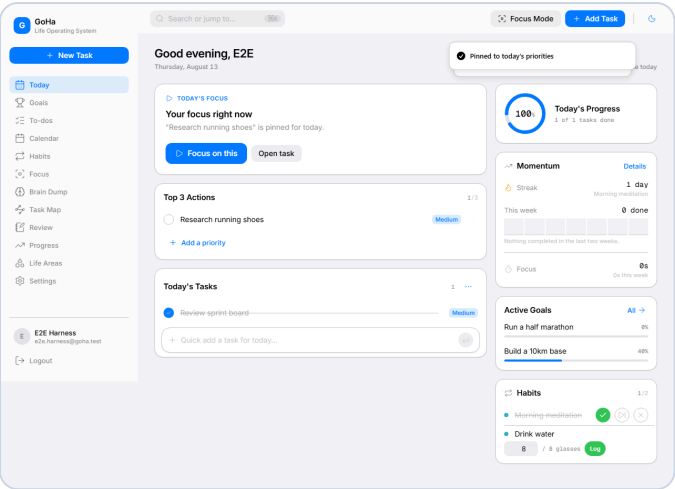
Accessibility gaps

- Custom Select places aria-activedescendant on an unfocused listbox while DOM focus remains on the trigger; active-option announcements may be unreliable.
- Tooltip trigger is not linked with aria-describedby.
- Mobile drawer lacks full focus trap/restoration; Task Map Import dialog does not reuse the accessible shared modal.
- Progress charts lack focusable values or an equivalent table/text summary.
- Goal timeframe tabs do not fully implement tab/tabpanel relationships and arrow-key behavior.
- Some Task Map controls are hover-only without focus-within/touch parity; dark tertiary/disabled copy needs a contrast pass.

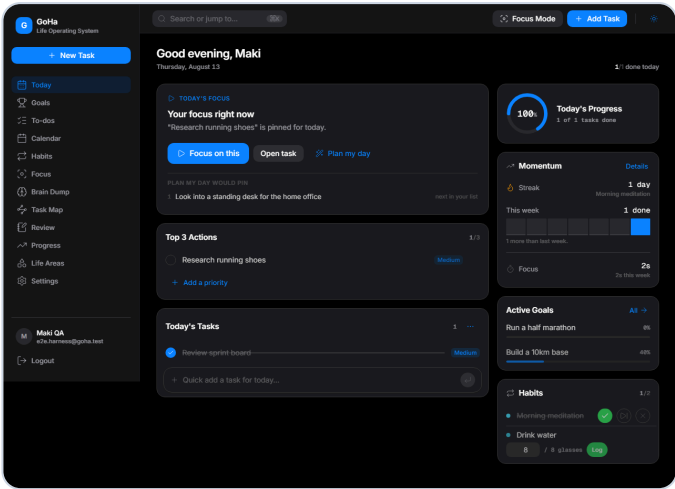
Existing visual evidence

Artifact caveat

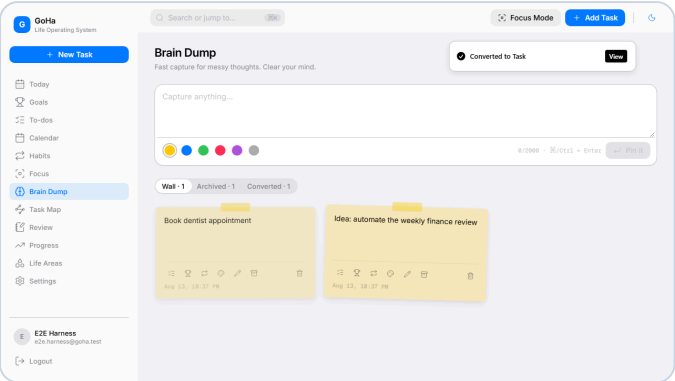
These are existing local QA screenshots, not fresh captures. The three mobile screenshots were taken before skeletons/animations settled and cannot prove final mobile spacing. They are included as historical visual evidence only.



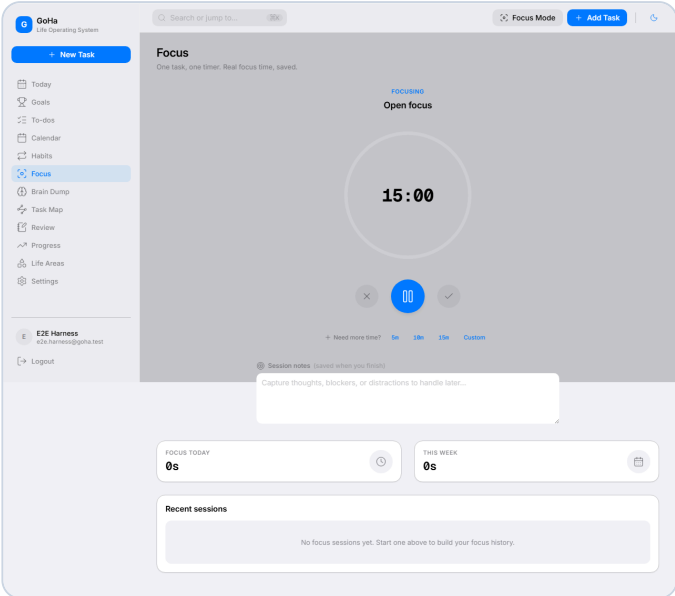
qa-screenshots/05-today.png Today light: dense 8/4 dashboard; toast overlaps the upper-right summary.



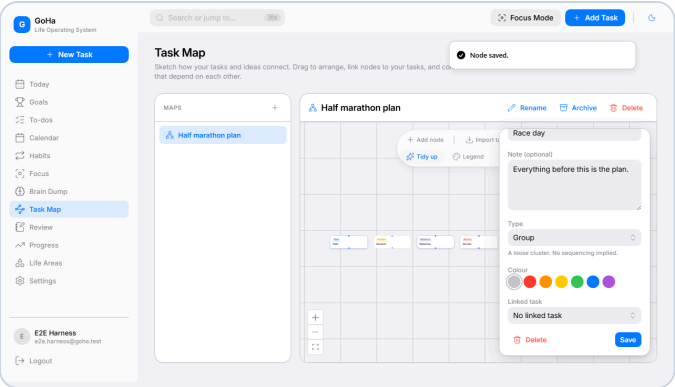
qa-screenshots/11-today-dark.png Today dark: cohesive true-black palette; faint tertiary copy.



qa-screenshots/06-brain-dump.png Brain Dump: shipped sticky-note wall is richer than the original list reference.

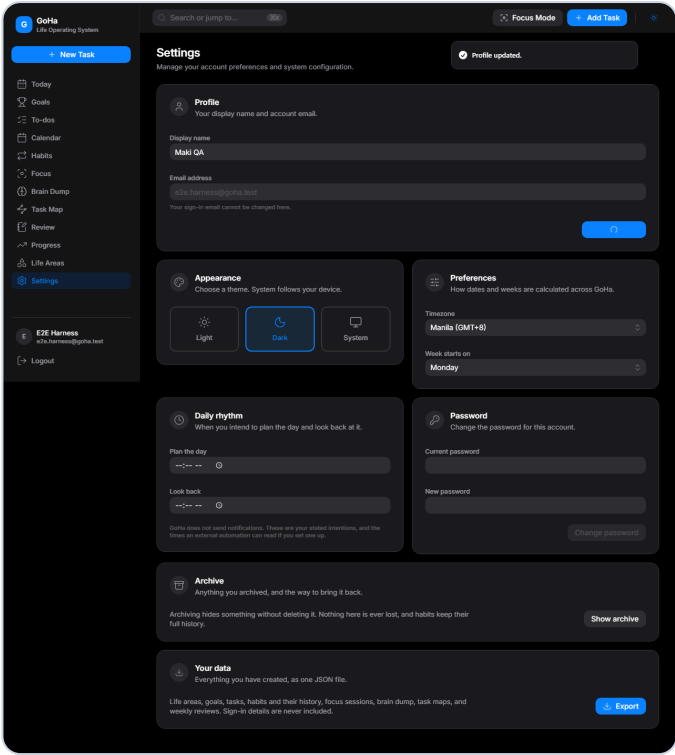


qa-screenshots/07-focus-running.png Focus running: cinematic dimming retained inside the full shell.



qa-screenshots/09-task-maps.png

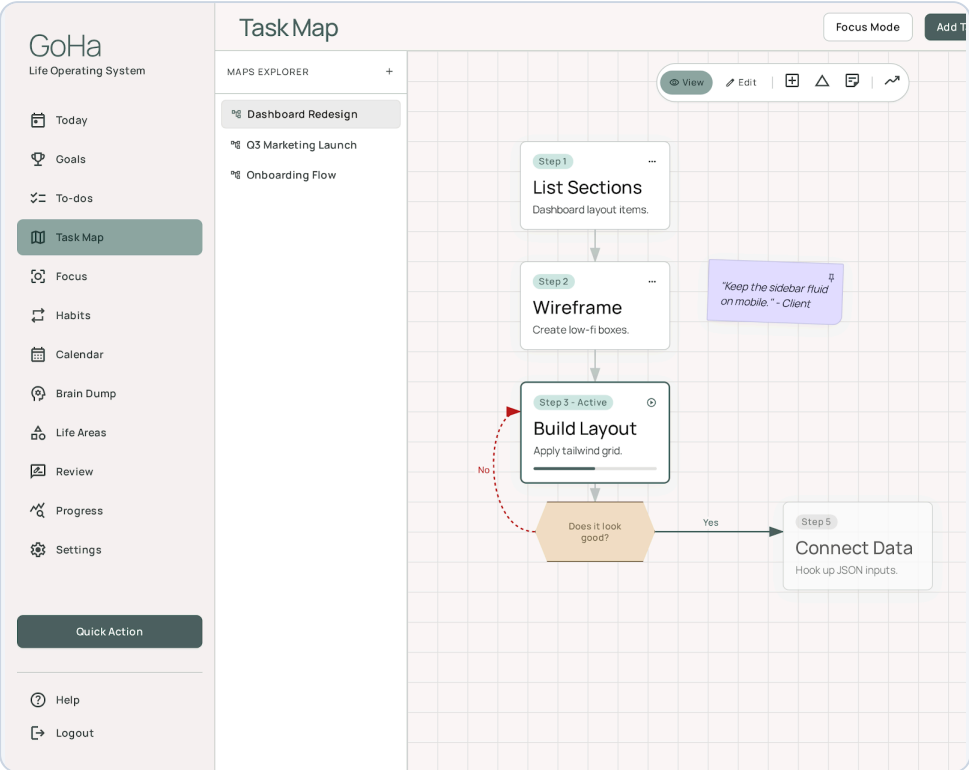
Task Maps: fitted nodes become tiny; inspector competes with canvas.



qa-screenshots/10-settings-dark.png

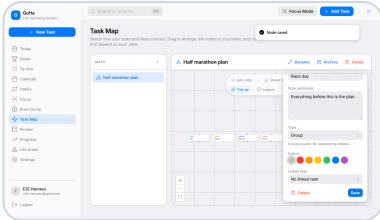
Settings dark: cohesive long-form settings and selected-state treatment.

Task Map reference divergence



design/stitch_goha_productivity_system_dashboard/task_map_explorer/screen.png

Historical Stitch reference: large legible nodes in a dependency flow.



qa-screenshots/09-task-maps.png

Current QA artifact: more capable editor, but low legibility after Tidy/fit.

Testing and quality evidence

The project has a meaningful historical test base, but no fresh tests were run for this audit and some collectors can report defects without failing.

Existing test inventory

LAYER	CURRENT WORKING TREE	STRENGTHS	GAPS
Vitest	15 files · 180 cases	Dates/timezones, streaks, progress, review, focus, goal progress, bucketing, validation, nav/classes, Task Map layout	No coverage provider/threshold; zero TSX/component, repository, action, auth, ownership, or migration tests
Playwright	8 spec files · 32 tests + auth setup	Persistence, task-goal flow, Brain conversion, stale session, Task Maps, subtasks/details, theme, mobile nav, keyboard	Chromium only; no retries; one worker; dev server default; gaps in Calendar/Review/Progress/Settings/security
Screenshots	15 ignored PNG artifacts	Feature, dark, mobile, focus, map evidence	No baseline comparison; mobile captured skeleton/mid-animation states
Historical gate	BUILD_PLAN reports 180/180 and 33/33	Repeated historical next start runs claimed	Not independently rerun in this read-only sweep

Unit-test distribution

AREA	CASES
Calendar/date engine	27
Focus calculations	14
Goal progress	11
Habit streak/view	20
Progress/review derivations	21
Settings/life-area validation	24
Task bucketing	13
Today scoring/signals	14
Navigation/class merge	11
Task Map layout	25

Test-system risks

- tests/cn.test.ts and tests/task-map-layout.test.ts contain 30 cases but are untracked.
- audit.spec collects console warnings/errors, uncaught exceptions, failed requests, 5xx responses, and slow timings but only prints them. qa.spec likewise records BUG/UX/console findings without a zero-defect assertion.
- trace:on-first-retry is inert because retries are zero. No standard failure video/screenshot policy, JUnit/HTML artifact publication, sharding, CI execution, Firefox/WebKit, axe scan, or visual-regression gate.
- qa.spec resets the test account before its suite using the normal database environment. E2E credential overrides in auth.setup are ignored by test-account.mts, so custom values can disagree.
- Calendar, Review, and Progress mainly receive route-load audit coverage, not persisted functional flows. Export, archives, password, palette, logout/relogin, second-owner rejection, and cross-user denial are not automated.

Static quality configuration

- tsconfig strict/noEmit/isolatedModules/bundler resolution plus Next plugin; ESLint Next core-web-vitals and TypeScript presets.
- No formatter/format-check, combined check script, coverage threshold, migration verification, bundle analysis, or performance budget.
- Testing Library packages are installed but unused. vite-tsconfig-paths is unused and brings deprecated unmaintained tsconfck.

Build, deployment, and operations

The codebase can build and test historically, but production readiness is not encoded as repeatable automation or runbooks.

Current configuration

- next.config.ts only pins the Turbopack root. No headers, CSP, poweredByHeader:false, standalone output, analyzer, explicit cache/observability policy, or robots/noindex policy for publicly deployed auth pages.
- .env.example documents DATABASE_URL, BETTER_AUTH_SECRET, BETTER_AUTH_URL only. E2E_PORT/BASE_URL/EMAIL/PASSWORD/NAME and dedicated database rules are absent.
- Four custom env parsers in Drizzle, migration, diagnose, and test-account tooling do not implement full Next env ordering, multiline/expansion, or all quote/comment semantics. @next/env would centralize this.
- No .github workflow, Vercel config, Dockerfile/compose, deployment manifest, Renovate/Dependabot, or security policy.
- No local PostgreSQL/container option or non-destructive seed/demo workflow exists; onboarding and tests depend on a Neon credential and the destructive E2E account harness.

Missing production capabilities

AREA	REPOSITORY STATE	NEEDED BASELINE
CI/CD	Absent	Frozen install, migration drift, typecheck, lint, unit+coverage, production build, safe production-mode E2E
Deploy sequencing	Absent	Backup/check → migrate → deploy compatible code → smoke/health → rollback decision
Health/readiness	Absent	Non-sensitive application and database readiness endpoint
Observability	console.error only	Central error aggregation, structured logs, environment/request correlation, alerts
Performance	No budget/monitoring	Bundle analysis, Web Vitals/RUM, Lighthouse or route budget
Database recovery	No repo runbook	Neon backup/PITR verification, export clarification, restore drill, owner recovery
Security posture	No explicit headers/scanning	CSP/headers, dependency scanning, secret scanning, incident process
Accessibility/visual	Manual/historical only	axe checks, settled screenshots, baseline comparisons, keyboard matrix

Historical bundle evidence

An existing ignored production artifact indicates very large uncompressed first-load route totals in historical build metadata (roughly 0.78–1.11 MB for feature routes, with Tasks/Habits/Goals among the largest). These numbers are not a fresh network transfer measurement and may include shared chunks, but they justify a real bundle analysis and budget.

Local artifact footprint

- Ignored .next occupies roughly 1.09 GB locally and is not a repository problem, but shows why builds were not rerun for a read-only audit.
- graphify-out is 61 tracked files / roughly 4.4 MB; design references 25 files / roughly 3.69 MB. Together they dominate tracked bytes.
- No Git remote or tags are configured. Unless an external backup exists, the repository itself has no remote code backup/release anchor.

Documentation and repository hygiene

The strongest current technical truth is scattered between code and chronological BUILD_PLAN entries; the official README and database reference are stale.

Documentation status

ARTIFACT	ASSESSMENT	KEY ISSUE
README.md	Create-next-app boilerplate	No GoHa purpose, architecture, environment, migrations, owner bootstrap, tests, deploy, recovery, or known issues; inaccurately implies Geist primary UI font
docs/DATABASE.md	Materially stale despite 'source of truth'	Documents only 0000/not applied; omits weekly reviews, subtasks, Brain color, map legend/note/new node types, migrations 0001–0010
docs/BUILD_PLAN.md	Rich chronological log	Old blocked/in-progress statements conflict with later completion; best current notes require manual reconciliation; correctly preserves Brain race
docs/GOHA_DESIGN_SPEC.md	Current visual source	Defines Apple-HIG/Liquid-Glass direction
docs/DESIGN_SYSTEM.md	Superseded legacy	Clearly labels former Nightshift direction
docs/STITCH_AUDIT.md + design/**	Historical/reference	External CDN assets in HTML; not production/offline assets
docs/GoHa-System-Guide.pdf	6 pages, Aug 12	Predates Phase 12/current Task Maps/QA; no tracked source
docs/GoHa-Automation-Guide.pdf	8 pages, Aug 12	Binary-only, dated, no generator/source

Generated and local artifacts

- graphify-out is dated August 12, describes 263 files and Phase 12 as not started, contains caches/intermediates, and embeds absolute C:\Users\Mark paths plus a user Python path.
- Its manifest indexes local .claude/settings.local.json. That settings file is untracked and not ignored and contains a machine-local tool permission.
- Five stock create-next-app SVGs and components/coming-soon.tsx are unused.
- No .gitattributes; Git warns about LF→CRLF conversion and cross-platform churn.
- QA screenshots and saved auth state are correctly ignored. Saved auth state remains sensitive even when ignored.

Recommended documentation set

- README: product purpose, stack, supported runtime, local setup, environment, bootstrap, commands, safe testing, docs map.
- CURRENT_STATE.md or product reference: active modules, implemented vs intentionally absent, known issues, canonical business rules.
- DATABASE.md regenerated from current schema/migrations with ownership/deletion/invariant notes.
- OPERATIONS.md: environments, migration/deploy sequence, backup/PITR and restore, health, logs, incident/account recovery.
- SECURITY.md: private-data posture, test-account guard, headers, vulnerability reporting, retention.
- Keep a source HTML/Markdown and generator next to every PDF for reproducibility.

19

Prioritized risk register

Severity reflects likely impact in this private single-owner product; findings are source-confirmed unless explicitly described as a risk dependent on deployment/runtime state.

SEVERITY	ID	FINDING	IMPACT / MECHANISM	EVIDENCE
CRITICAL	R-01	Migration 0010 deployment drift	Application/schema/journal require new enum values and node note, but SQL and snapshot are untracked. A clean clone can fail at runtime.	db/migrations/0010_jazzy_husk.sql · db/migrations/meta/0010_snapshot.json
CRITICAL	R-02	Destructive E2E can target the owner database	Playwright and test-account reset use whichever DATABASE_URL the normal environment supplies; there is no dedicated-test marker/host/name guard. Known credentials and direct inserts bypass single-owner policy.	playwright.config.ts:37 · e2e/qa.spec.ts:27 · scripts/test-account.mts
HIGH	R-03	Legacy migration data loss/misclassification	0002 defaults all prior habit rows to done before 0003 drops completed; 0005 does not backfill converted_task_id before 0006 drops it. Actual exposure depends on deployment history.	db/migrations/0002...0006
HIGH	R-04	Export is not a complete backup	It omits graph nodes/edges, daily priorities, goal progress history, active focus, inactive schedules, and data outside hard caps/ranges. No import/restore exists.	app(app)/settings/export-actions.ts
HIGH	R-05	Review state can cross weeks	Local state is initialized once from review props; query navigation can preserve prior-week prose/rating and save it against the new week.	components/review/review-view.tsx:84-87
HIGH	R-06	Numeric habit results contradict across modules	Below-target numeric entries are stored done; Habits/Today derive partial, while Calendar and wide-window Progress/Review count raw done.	lib/habit-streaks.ts · app(app)/calendar/page.tsx · lib/progress.ts
HIGH	R-07	Calendar silently truncates habit/focus history	Only ±62 days are fetched once, yet month navigation is unlimited. Due-only task placement also falls back to Manila.	app(app)/calendar/page.tsx:19-33 · components/calendar/calendar-view.tsx

SEVERITY	ID	FINDING	IMPACT / MECHANISM	EVIDENCE
HIGH	R-08	Single-owner and active-row concurrency races	Read-then-create signup, focus close-then-start, schedule creation, priorities, goal cycles, and extensions lack database serialization/version constraints.	lib/auth.ts:48-61 · db/repositories/focus.ts
HIGH	R-09	Protocol-relative redirect accepted	Checking only startsWith('/') accepts //attacker.example in both server and client redirect handling.	app/(auth)/login/page.tsx:17 · components/auth/auth-form.tsx:54
HIGH	R-10	Known Brain Dump rapid-capture dead state	Repeated capture under roughly 400 ms can leave aria-busy and disabled state permanently pending until reload; QA intentionally waits around it.	docs/BUILD_PLAN.md:613-624 · e2e/qa.spec.ts:400-415
HIGH	R-11	Audit E2E can pass while reporting runtime bugs	Console/error/5xx/failed-request and QA BUG collectors print findings but do not assert an empty problem list.	e2e/audit.spec.ts:211 · e2e/qa.spec.ts:658
HIGH	R-12	Task Map archive/delete integrity	Archived maps remain editable; node/edge deletion is immediate with neither confirmation nor rollback; duplicate task links can distort progress.	components/task-maps/task-maps-workspace.tsx · components/task-maps/flow-canvas.tsx
MEDIUM	R-13	Ownership is not a database invariant	Actions are generally owner-scoped, but FKs do not prove child/parent owner equality and there is no RLS. Scripts or future bugs can create cross-owner graphs.	db/schema/**
MEDIUM	R-14	Non-atomic multi-write workflows	Habit+schedule, goal+journal, focus replacement, plan-day, task import, and position updates may partially succeed under Neon HTTP transaction:false.	db/client.ts · lib/auth.ts
MEDIUM	R-15	Timezone contract is inconsistent	Focus sessionId, export range/name, some Today/Tasks sorting, and Calendar due-date grouping use Manila defaults instead of the saved timezone. DST gap/ambiguity round trips are not validated.	db/repositories/focus.ts:52 · app/(app)/settings/export-actions.ts:37
MEDIUM	R-16	Production operation is not encoded	No CI/CD, safe migration sequence, health endpoint, central observability, backup/restore runbook, security-header policy, performance budget, or dependency security automation.	next.config.ts · README.md

SEVERITY	ID	FINDING	IMPACT / MECHANISM	EVIDENCE
MEDIUM	R-17	Focus draft/input safety	Invalid custom text can leave a previous duration startable; notes save only on finish; discard is unconfirmed; session has no notification/auto-end.	<code>components/focus/**</code>
MEDIUM	R-18	Progress periods and accessibility	Comparison windows are unequal, habit rate compares dissimilar horizons, and visual charts lack keyboard/tabular equivalents. Route also lacks loading/error boundaries.	<code>app/(app)/progress/page.tsx · components/charts/*</code>
MEDIUM	R-19	Environment/tooling contract is incomplete	E2E variables are undocumented, four parsers diverge from Next env semantics, Node/pnpm are unpinned, and .mts tooling may fail on a Next-supported Node 20 runtime.	<code>.env.example · package.json</code>
MEDIUM	R-20	Documentation and artifact drift	README is boilerplate; DATABASE is stale; BUILD_PLAN mixes old and current truth; Graphify and existing PDFs predate current work and are not reproducible from source.	<code>README.md · docs/** · graphify-out/**</code>
LOW	R-21	Repository hygiene	Local settings are untracked/not ignored, no .gitattributes, unused packages/assets remain, no remote/tags, and generated analysis caches contain absolute user paths.	<code>.claude/settings.local.json · graphify-out/.graphify_root</code>

Severity is not a claim that exploitation or data loss has occurred. Migration application, hosted headers, external backups, and live production behavior were not queried. Critical items are unsafe preconditions for deployment or destructive commands.

Remediation roadmap

A practical order that first protects data and deployment, then fixes correctness, then establishes repeatable operations and product refinement.

P0 — before deployment or another destructive test run

1. Track migration 0010 and snapshot; verify journal/schema consistency and applied production state; test a clean migration chain on disposable PostgreSQL.
2. Make all E2E/test-account commands require a dedicated database URL plus an explicit marker/confirmation. Reject production-like hosts/names. Make the helper honor E2E_* credentials.
3. Replace the export completeness claim. Add nodes, edges, priorities, goal journal, active focus, all schedules, explicit pagination/caps, a manifest/schema version, and a tested import/restore path—or label it partial.
4. Add data-correct backfill migrations/verification for legacy habit status and Brain conversion provenance before touching any deployment that passed through those versions.

P1 — correctness and security

1. Key/reset Review draft state by weekStart and add dirty-navigation protection plus a regression E2E.
2. Normalize habit outcome once and use it in all five consumers; define exact X/week and monthly occurrence semantics.
3. Make Calendar query by viewed month or preload with explicit range UI; pass required timezone; filter day detail and reset selection on Today.
4. Fix Brain Dump transition lifecycle and regression-test rapid captures; truncate/validate conversion titles at target limits.
5. Restrict redirects to one-slash same-origin paths. Add singleton owner protection and partial unique indexes for active focus/schedule where feasible.
6. Guard archived Task Maps, add confirmation/rollback for graph deletion, de-duplicate linked tasks in progress, and constrain layout output.
7. Introduce safer atomic operations: Neon driver/database procedure/CTE patterns or compensating logic for habit+schedule, goal+journal, focus replacement, planning, and imports.

P2 — production baseline

1. Pin Node and pnpm; centralize env parsing/validation; document E2E and production origin/HTTPS contracts.
2. Add CI: frozen install, migration drift/apply test, typecheck, lint, unit coverage, production build, safe E2E with failure artifacts and runtime-error assertions.
3. Add security headers/CSP, poweredByHeader:false, dependency scanning, health/readiness, central error/logging, Web Vitals, and bundle/performance budgets.
4. Document and test backup/PITR/restore, migration-before-deploy, rollback, owner recovery, retention, and incident response.
5. Add component/action/repository/authorization/migration tests; functional Calendar/Review/Progress/Settings coverage; Firefox/WebKit, axe, and settled visual baselines.

P3 — product and maintainability

1. Add entity deep links and let search/Calendar/Map/Brain/Review open the exact record.
2. Improve Focus notifications/auto-end/draft note persistence and confirm discard.
3. Implement or remove unsupported Life Score/importance claims, monthly schedule fields, free-text priorities, and unused estimateMinutes.
4. Refresh README, DATABASE, current-state and operations docs; reduce stale Graphify caches; ignore local settings; add .gitattributes; remove unused dependencies/assets.
5. Perform a dark-mode contrast pass, Task Map legibility redesign, Progress accessible table alternatives, and final mobile capture at settled states.

APPENDIX A

Server Action catalog (62)

Every exported async Server Action discovered under `app/**/actions.ts` in the inspected working tree.

FILE	COUNT	EXPORTED ACTIONS
<code>app/(app)/brain-dump/actions.ts</code>	7	<code>captureBrainDumpItemAction</code> · <code>updateBrainDumpItemAction</code> · <code>setBrainDumpColorAction</code> · <code>archiveBrainDumpItemAction</code> · <code>restoreBrainDumpItemAction</code> · <code>deleteBrainDumpItemAction</code> · <code>convertBrainDumpItemAction</code>
<code>app/(app)/focus/actions.ts</code>	6	<code>startFocusSessionAction</code> · <code>pauseFocusSessionAction</code> · <code>resumeFocusSessionAction</code> · <code>extendFocusSessionAction</code> · <code>endFocusSessionAction</code> · <code>discardFocusSessionAction</code>
<code>app/(app)/goals/actions.ts</code>	3	<code>createGoalAction</code> · <code>updateGoalAction</code> · <code>archiveGoalAction</code>
<code>app/(app)/habits/actions.ts</code>	5	<code>createHabitAction</code> · <code>updateHabitAction</code> · <code>archiveHabitAction</code> · <code>logHabitEntryAction</code> · <code>clearHabitEntryAction</code>
<code>app/(app)/life-areas/actions.ts</code>	4	<code>createLifeAreaAction</code> · <code>updateLifeAreaAction</code> · <code>archiveLifeAreaAction</code> · <code>restoreLifeAreaAction</code>
<code>app/(app)/review/actions.ts</code>	2	<code>saveWeeklyReviewAction</code> · <code>reopenWeeklyReviewAction</code>
<code>app/(app)/search-actions.ts</code>	1	<code>loadCommandIndexAction</code>
<code>app/(app)/settings/actions.ts</code>	4	<code>updateProfileAction</code> · <code>updateThemeAction</code> · <code>updateRhythmAction</code> · <code>updatePreferencesAction</code>
<code>app/(app)/settings/archive-actions.ts</code>	2	<code>listArchivedAction</code> · <code>restoreArchivedAction</code>
<code>app/(app)/settings/export-actions.ts</code>	1	<code>exportMyDataAction</code>
<code>app/(app)/task-maps/actions.ts</code>	16	<code>createTaskMapAction</code> · <code>updateTaskMapAction</code> · <code>archiveTaskMapAction</code> · <code>restoreTaskMapAction</code> · <code>deleteTaskMapAction</code> · <code>saveViewportAction</code> · <code>addNodeAction</code> · <code>updateNodeAction</code> · <code>moveNodesAction</code> · <code>deleteNodeAction</code> · <code>saveLegendAction</code> · <code>importTasksAction</code> · <code>updateEdgeLabelAction</code> · <code>layoutMapAction</code> · <code>addEdgeAction</code> · <code>deleteEdgeAction</code>
<code>app/(app)/tasks/actions.ts</code>	8	<code>createTaskAction</code> · <code>updateTaskAction</code> · <code>completeTaskAction</code> · <code>reopenTaskAction</code> · <code>cancelTaskAction</code> · <code>createSubtaskAction</code> · <code>saveCompletionNoteAction</code> · <code>deleteTaskAction</code>
<code>app/(app)/today/actions.ts</code>	3	<code>addDailyPriorityAction</code> · <code>planMyDayAction</code> · <code>removeDailyPriorityAction</code>

The catalog is mechanically derived from the working tree. It excludes Better Auth Route Handler exports and repository functions because they are not Server Actions.

APPENDIX B

Complete tracked-file inventory

All 358 tracked paths plus the six relevant pre-report untracked paths. Generated dependency/build internals are intentionally summarized rather than listed.

Pre-report untracked paths

- .claude/settings.local.json
- db/migrations/0010_jazzy_husk.sql
- db/migrations/meta/0010_snapshot.json
- lib/task-map-layout.ts
- tests/cn.test.ts
- tests/task-map-layout.test.ts

(repository root) 18 tracked

.env.example	package.json
.gitignore	playwright.config.ts
AGENTS.md	pnpm-lock.yaml
CLAUDE.md	pnpm-workspace.yaml
README.md	postcss.config.mjs
components.json	proxy.ts
drizzle.config.ts	tsconfig.json
eslint.config.mjs	vitest.config.ts
next.config.ts	vitest.setup.ts

app 56 tracked

app/(app)/brain-dump/actions.ts	app/(app)/life-areas/error.tsx
app/(app)/brain-dump/error.tsx	app/(app)/life-areas/loading.tsx
app/(app)/brain-dump/loading.tsx	app/(app)/life-areas/page.tsx
app/(app)/brain-dump/page.tsx	app/(app)/progress/page.tsx
app/(app)/calendar/error.tsx	app/(app)/review/actions.ts
app/(app)/calendar/loading.tsx	app/(app)/review/error.tsx
app/(app)/calendar/page.tsx	app/(app)/review/loading.tsx
app/(app)/focus/actions.ts	app/(app)/review/page.tsx
app/(app)/focus/error.tsx	app/(app)/search-actions.ts
app/(app)/focus/loading.tsx	app/(app)/settings/actions.ts
app/(app)/focus/page.tsx	app/(app)/settings/archive-actions.ts
app/(app)/goals/actions.ts	app/(app)/settings/error.tsx
app/(app)/goals/error.tsx	app/(app)/settings/export-actions.ts
app/(app)/goals/loading.tsx	app/(app)/settings/loading.tsx
app/(app)/goals/page.tsx	app/(app)/settings/page.tsx
app/(app)/habits/actions.ts	app/(app)/task-maps/actions.ts
app/(app)/habits/error.tsx	app/(app)/task-maps/error.tsx
app/(app)/habits/loading.tsx	app/(app)/task-maps/loading.tsx
app/(app)/habits/page.tsx	app/(app)/task-maps/page.tsx
app/(app)/layout.tsx	app/(app)/tasks/actions.ts
app/(app)/life-areas/actions.ts	app/(app)/tasks/error.tsx

app/(app)/tasks/loading.tsx
app/(app)/tasks/page.tsx
app/(app)/today/actions.ts
app/(app)/today/error.tsx
app/(app)/today/loading.tsx
app/(app)/today/page.tsx
app/(auth)/layout.tsx

components 73 tracked

components/auth/auth-form.tsx
components/auth/logout-button.tsx
components/brain-dump/brain-dump-item.tsx
components/brain-dump/brain-dump-view.tsx
components/calendar/calendar-view.tsx
components/celebration.tsx
components/charts/bar-series.tsx
components/charts/habit-heatmap.tsx
components/coming-soon.tsx
components/date-rollover-refresh.tsx
components/empty-state.tsx
components/focus/duration-picker.tsx
components/focus/extend-control.tsx
components/focus/focus-stats.tsx
components/focus/focus-view.tsx
components/goals/goal-card.tsx
components/goals/goal-detail-panel.tsx
components/goals/goal-form-modal.tsx
components/goals/goals-view.tsx
components/habits/habit-form-modal.tsx
components/habits/habit-log-control.tsx
components/habits/habits-view.tsx
components/habits/habits-week-grid.tsx
components/life-areas/icon.tsx
components/life-areas/life-area-card.tsx
components/life-areas/life-area-form-modal.tsx
components/life-areas/life-areas-view.tsx
components/motion-config.tsx
components/page-header.tsx
components/progress/progress-view.tsx
components/review/review-view.tsx
components/settings/settings-view.tsx
components/shell/app-header.tsx
components/shell/app-sidebar.tsx
components/shell/brand.tsx
components/shell/command-palette.tsx
components/shell/mobile-bottom-nav.tsx

db 51 tracked

db/client.ts
db/index.ts
db/migrate.mts
db/migrations/0000_init_full_schema.sql
db/migrations/0001_tasks_status_priority_feedback.sql
db/migrations/0002_habit_entry_status_add.sql

app/(auth)/login/page.tsx
app/(auth)/register/page.tsx
app/api/auth/[...all]/route.ts
app/favicon.ico
app/globals.css
app/layout.tsx
app/page.tsx

components/shell/mobile-header.tsx
components/shell/nav-link.tsx
components/shell/theme-toggle.tsx
components/shell/user-badge.tsx
components/task-maps/flow-canvas.tsx
components/task-maps/task-maps-workspace.tsx
components/tasks/completion-note-modal.tsx
components/tasks/task-calendar.tsx
components/tasks/task-card.tsx
components/tasks/task-detail-panel.tsx
components/tasks/task-form-modal.tsx
components/tasks/tasks-view.tsx
components/theme-provider.tsx
components/theme-sync.tsx
components/today/active-goals-card.tsx
components/today/brain-card.tsx
components/today/momentum-card.tsx
components/today/progress-ring.tsx
components/today/quick-add-task.tsx
components/today/task-checklist-item.tsx
components/today/today-habits.tsx
components/today/today-view.tsx
components/today/top-priorities.tsx
components/ui/button.tsx
components/ui/card.tsx
components/ui/checkbox.tsx
components/ui/detail-panel.tsx
components/ui/dropdown.tsx
components/ui/input.tsx
components/ui/modal.tsx
components/ui/progress.tsx
components/ui/segmented-control.tsx
components/ui/select.tsx
components/ui/sonner.tsx
components/ui/textarea.tsx
components/ui/tooltip.tsx

db/migrations/0003_habit_entry_drop_completed.sql
db/migrations/0004_focus_pause_tracking.sql
db/migrations/0005_brain_dump_converted_add.sql
db/migrations/0006_brain_dump_drop_task_id.sql
db/migrations/0007_busy_whizzer.sql
db/migrations/0008_slow_silhouette.sql

db/migrations/0009_shiny_vengeance.sql
db/migrations/meta/0000_snapshot.json
db/migrations/meta/0001_snapshot.json
db/migrations/meta/0002_snapshot.json
db/migrations/meta/0003_snapshot.json
db/migrations/meta/0004_snapshot.json
db/migrations/meta/0005_snapshot.json
db/migrations/meta/0006_snapshot.json
db/migrations/meta/0007_snapshot.json
db/migrations/meta/0008_snapshot.json
db/migrations/meta/0009_snapshot.json
db/migrations/meta/_journal.json
db/repositories/brain-dump.ts
db/repositories/daily-priorities.ts
db/repositories/focus.ts
db/repositories/goals.ts
db/repositories/habits.ts
db/repositories/index.ts
db/repositories/life-areas.ts
db/repositories/reviews.ts

design 25 tracked

design/stitch_goha_productivity_system_dashboard/brain_dump_quick_capture/code.html
design/stitch_goha_productivity_system_dashboard/brain_dump_quick_capture/screen.png
design/stitch_goha_productivity_system_dashboard/calendar_schedule/code.html
design/stitch_goha_productivity_system_dashboard/calendar_schedule/screen.png
design/stitch_goha_productivity_system_dashboard/focus_session/code.html
design/stitch_goha_productivity_system_dashboard/focus_session/screen.png
design/stitch_goha_productivity_system_dashboard/goals_management/code.html
design/stitch_goha_productivity_system_dashboard/goals_management/screen.png
design/stitch_goha_productivity_system_dashboard/habit_tracker/code.html
design/stitch_goha_productivity_system_dashboard/habit_tracker/screen.png
design/stitch_goha_productivity_system_dashboard/life_areas_dashboard/code.html
design/stitch_goha_productivity_system_dashboard/life_areas_dashboard/screen.png
design/stitch_goha_productivity_system_dashboard/life_score_progress_analytics/code.html

docs 7 tracked

docs/BUILD_PLAN.md
docs/DATABASE.md
docs/DESIGN_SYSTEM.md
docs/GOHA_DESIGN_SPEC.md

e2e 9 tracked

e2e/audit.spec.ts

db/repositories/settings.ts
db/repositories/task-maps.ts
db/repositories/tasks.ts
db/repositories/users.ts
db/schema/_shared.ts
db/schema/auth.ts
db/schema/brain-dump.ts
db/schema/daily-priorities.ts
db/schema/enums.ts
db/schema/focus.ts
db/schema/goals.ts
db/schema/habits.ts
db/schema/index.ts
db/schema/life-areas.ts
db/schema/reviews.ts
db/schema/settings.ts
db/schema/task-maps.ts
db/schema/tasks.ts
db/types.ts

design/stitch_goha_productivity_system_dashboard/life_score_progress_analytics/screen.png
design/stitch_goha_productivity_system_dashboard/serene_lifecycle/DESIGN.md
design/stitch_goha_productivity_system_dashboard/settings_preferences/code.html
design/stitch_goha_productivity_system_dashboard/settings_preferences/screen.png
design/stitch_goha_productivity_system_dashboard/task_map_explorer/code.html
design/stitch_goha_productivity_system_dashboard/task_map_explorer/screen.png
design/stitch_goha_productivity_system_dashboard/to_do_lists/code.html
design/stitch_goha_productivity_system_dashboard/to_do_lists/screen.png
design/stitch_goha_productivity_system_dashboard/today_dashboard/code.html
design/stitch_goha_productivity_system_dashboard/today_dashboard/screen.png
design/stitch_goha_productivity_system_dashboard/weekly_review_reflection/code.html
design/stitch_goha_productivity_system_dashboard/weekly_review_reflection/screen.png

docs/GoHa-Automation-Guide.pdf
docs/GoHa-System-Guide.pdf
docs/STITCH_AUDIT.md

e2e/auth.setup.ts

e2e/brain-dump.spec.ts

e2e/life-areas.spec.ts

e2e/qa.spec.ts

e2e/stale-session.spec.ts

e2e/task-details.spec.ts

e2e/task-maps.spec.ts

e2e/tasks.spec.ts

graphify-out 61 tracked

graphify-out/.graphify_labels.json

graphify-out/.graphify_python

graphify-out/.graphify_root

graphify-out/GRAPH_REPORT.md

graphify-out/cache/ast/v0.9.40/159ff1ac1f176e1a9d83c92e1132a5a9812fcbc84768f811b48ba88a5a46259a7.json

graphify-out/cache/ast/v0.9.40/62e09e3e91a5859a0ed4fec076013973848bd8f737a61bec2d164507d906c939.json

graphify-out/cache/ast/v0.9.40/6ce48f9ecfc524a7646d5d5e1a0eb9269150b3764e310770c4c008da84737145.json

graphify-out/cache/ast/v0.9.40/73cf039b62812217cd944b55aee3aa1a5168dbbdec5b9eb2cf3bb0f69c8b9f21.json

graphify-out/cache/ast/v0.9.40/821ba8f3280a6a8be155ef5fe5c74c03b6f3a5a5ed1a614c889f9a8229967c7d.json

graphify-out/cache/ast/v0.9.40/94791944df7f5bd0dff609a30fad4d137b38bb56e6c2fd245420757895b2e10d.json

graphify-out/cache/ast/v0.9.40/9c2fe9ea886f0e1477b54a9ea03a4a13ea6330ef4850bdb6689173d0b35017ab.json

graphify-out/cache/ast/v0.9.40/9da5dda6b95bf789237cdde45b6d471064a6067a9ab6b262e7c25f8cd0b98d72.json

graphify-out/cache/ast/v0.9.40/b0824d400a5c374f5f5dfbe7f51bd0bc1e1929c9fb0d226a038d4daa6d3bbed4.json

graphify-out/cache/ast/v0.9.40/c49ae56747e6b26ca8f22bd3ad1a1ece607ec7f8566bd8cf7c589d823fc652a1.json

graphify-out/cache/ast/v0.9.40/d524061f92850225f80bf16cf38111ee3430932c85286e5a3996d4c74f905e7c.json

graphify-out/cache/ast/v0.9.40/fe9eb7a8a67421c7d53e47c094c83a4c8dedb18f6392c97fc99fd7f60b49627e.json

graphify-out/cache/last_query_stamp

graphify-out/cache/semantic/pd5fd89c46bb5/04131dae19e730ec8cc28a49f989f1081a7abba45d8009bec4b14c0512680906.json

graphify-out/cache/semantic/pd5fd89c46bb5/09e4e7ac0e8a9d90a77418efa4de17c6999b5258e2615708cb26684a9696bcc2.json

graphify-out/cache/semantic/pd5fd89c46bb5/0b44daaa94766d385f056c95e8aabb30ad6e49ef3aa9c7eb9e32606055f4eaf2.json

graphify-out/cache/semantic/pd5fd89c46bb5/0bce824ceb9195a5b8da1ac9cc58aa54bdf98e245ad79d7286403b873ac13e99.json

graphify-out/cache/semantic/pd5fd89c46bb5/13609492b90c2eaea49b377d6eeefa81c2aaf82445cdcb1c6b9ae766734e4f6da.json

graphify-out/cache/semantic/pd5fd89c46bb5/16f1a413d1f040c82e545b8099765926a55e54c7e73c9220fc53529bd85fff0c.json

graphify-out/cache/semantic/pd5fd89c46bb5/1c07fe14390e84fd833eb1068fe9837ac166b343f8ba976d2b2bf30e8844130c.json

graphify-out/cache/semantic/pd5fd89c46bb5/26374b79ad46171d16d961d2fede50787d8ce2b4e505298ad87bea598af95a16.json

graphify-out/cache/semantic/pd5fd89c46bb5/289c39cb258ce9facc3435b0824b0e3c3da500d9ae96cc35252cd23b346a5a2b.json

graphify-out/cache/semantic/pd5fd89c46bb5/2a6f285a1edf78974384141c6d72ef402520f941efd9733bd1c81fab17676112.json

graphify-out/cache/semantic/pd5fd89c46bb5/413298aedfae295d2183ae4a9f02c72710f85021c650e89cab93a5cdc8785e93.json

graphify-out/cache/semantic/pd5fd89c46bb5/415682a44df88c5e528db8bca99276f1ec5f90d4e7fb73d1772756b6b4eb4fee.json

graphify-out/cache/semantic/pd5fd89c46bb5/428867838da76f6fbe83b733b3b0b2d4276711348ec8e2d557d10ef105c62287.json

graphify-out/cache/semantic/pd5fd89c46bb5/437b77f8dc05e33b5231d799f3e8d6fb892b865a7e4b81013621a0672900b704.json

graphify-out/cache/semantic/pd5fd89c46bb5/52163ecb5257da6ecb3ddf8967f0d51a0da93d79d48caa09e9d307ea1c1d3f17.json

graphify-out/cache/semantic/pd5fd89c46bb5/58d233ee9a0b0231d74f31667bfd20bc006b78d64d40f6521293bd71f06d435e.json

graphify-out/cache/semantic/pd5fd89c46bb5/5de47413b8cba30d5c35d1d68455051b45a44d482d29993de0fa13eb370e280.json

graphify-out/cache/semantic/pd5fd89c46bb5/6a7cf7d7e93ffc65b784bbaf51e422dfdb02690ecab68422c4ccdca063da84fb.json

graphify-out/cache/semantic/pd5fd89c46bb5/8009e2f6e324c325a0519838a029a53e489b5c4e39567ccb82300fca5d5aee39.json

graphify-out/cache/semantic/pd5fd89c46bb5/8a010345595bfa29c79be185d7bd6cfce55dabb019f2e1af2b4f2a371d99bf11.json

graphify-out/cache/semantic/pd5fd89c46bb5/8c99baedee05ada37bd392e179350c9a8e893c25c08cb2e1a8ec25f4f8a33868.json

graphify-out/cache/semantic/pd5fd89c46bb5/93ae74a50fdc70d076e02ad248b23eadb8397a3d99a53e8bffc2c442190c68b.json

graphify-out/cache/semantic/pd5fd89c46bb5/95197a793e0b5764060c6978b97b74bfaf72f0473033a65e09bde6fc0d4e3d3b.json

graphify-out/cache/semantic/pd5fd89c46bb5/97d53baf901a03d27cc913d47d8d8aa9d024a3d29289ab69c1fb79dc6cdba50f.json

graphify-out/cache/semantic/pd5fd89c46bb5/9903fac41fc0ae44a8b4a4ebeaaf1e6298896eef3b2d1e50d58bf9b89afc946b.json

graphify-out/cache/semantic/pd5fd89c46bb5/9caa34ac82b48d59fd2add4c0dbbf6fceaf3c3904254c3ccd89dd00644b375ad.json

graphify-out/cache/semantic/pd5fd89c46bb5/a36a5131f0f33d427ea2fbfcf07489db4b628ece6dbd5763674080f03c01c8c4.json
graphify-out/cache/semantic/pd5fd89c46bb5/aba5f85415e644173c46c2c7fc02752da92798ef4a8eb2ba8409bbb3edb336f3.json
graphify-out/cache/semantic/pd5fd89c46bb5/ae4281695c73ae91f46a21d75e25007811077ead4794b9137fec9f08f6734193.json
graphify-out/cache/semantic/pd5fd89c46bb5/b021811934b2dcf6880c17073cc4722341d0777b60bf29ac493f2a87cff09a82.json
graphify-out/cache/semantic/pd5fd89c46bb5/b488eaf6f13cffe5f94f251b1cd0ba03cc9906daaa77e6083194f76306454b7f.json
graphify-out/cache/semantic/pd5fd89c46bb5/b9c528141d39fc78e85bf37c539d8822d4e98431469edcb7ffde5e49224f4e2a.json
graphify-out/cache/semantic/pd5fd89c46bb5/ce828de76b1f7733c27cee93a134667bd043bf0e1ae602595201418f04f5f774.json
graphify-out/cache/semantic/pd5fd89c46bb5/d10932e2577d5c9a1aad47ac8c100ace5260237cf6379f5c889c5b1eee72a773.json

lib 37 tracked

lib/auth-client.ts
lib/auth.ts
lib/brain-dump.ts
lib/calendar.ts
lib/date.ts
lib/focus-reconcile.ts
lib/focus.ts
lib/goal-progress.ts
lib/goals.ts
lib/habit-streaks.ts
lib/habit-view.ts
lib/habits.ts
lib/life-areas.ts
lib/milestones.ts
lib/motion.ts
lib/nav.ts
lib/progress.ts
lib/review.ts
lib/session.ts

public 5 tracked

public/file.svg
public/globe.svg
public/next.svg

scripts 2 tracked

scripts/diagnose.mts

stores 1 tracked

stores/focus-timer.ts

graphify-out/cache/semantic/pd5fd89c46bb5/da6f586c05173cbe29b7f719ab6bfaec73c83e77c56f11a7fcb09dab61f94fb3.json
graphify-out/cache/semantic/pd5fd89c46bb5/dd579ec801cede14607350aa14a79bf784424d7144e1b63dbcbc4b9e32c12474.json
graphify-out/cache/semantic/pd5fd89c46bb5/efa7a399835a6b46bb4aa8c2dcac734e15fdd0428503fd47f112c4ff21c206b6.json
graphify-out/cache/semantic/pd5fd89c46bb5/f56c045273213f32f4acd96e98dfb664cfc7c3f71d6775317d19c60d38439655.json
graphify-out/cache/semantic/pd5fd89c46bb5/face358738bdc0eed23e38c566120f88a41ad892a20335a78931931a42db2431.json
graphify-out/cache/stat-index.json
graphify-out/cost.json
graphify-out/graph.html
graphify-out/graph.json
graphify-out/manifest.json

lib/task-buckets.ts
lib/task-maps.ts
lib/tasks.ts
lib/timezones.ts
lib/today-brain.ts
lib/today.ts
lib/use-media-query.ts
lib/use-mounted.ts
lib/use-now.ts
lib/user-settings.ts
lib/utils.ts
lib/validations/brain-dump.ts
lib/validations/goal.ts
lib/validations/habit.ts
lib/validations/life-area.ts
lib/validations/settings.ts
lib/validations/task-maps.ts
lib/validations/task.ts

public/vercel.svg
public/window.svg

scripts/test-account.mts

tests 13 tracked

tests/calendar.test.ts	tests/progress.test.ts
tests/date.test.ts	tests/review.test.ts
tests/focus.test.ts	tests/settings-schema.test.ts
tests/goal-progress.test.ts	tests/shell.test.ts
tests/habit-streaks.test.ts	tests/task-buckets.test.ts
tests/habit-view.test.ts	tests/today-brain.test.ts
tests/life-area-schema.test.ts	

The generated PDF itself is the authorized output and is not part of the pre-report baseline. node_modules, .next, .git internals, ignored Playwright artifacts, ignored QA screenshots, and saved auth state are not tracked-file inventory entries; they are described in the operations/design sections where relevant.

APPENDIX C

Glossary and ownership map

A concise index of system terms and the files that define them.

TERM	MEANING	PRIMARY SOURCE
Canonical row	The durable Task/Goal/Habit/etc. record reused across modules rather than a dashboard copy.	db/schema/**
Owner	The single intended Better Auth user. All domain rows still carry userId.	lib/auth.ts · lib/session.ts
Effective task date	scheduledDate when present, otherwise dueAt converted into a timezone-local ISO date.	lib/tasks.ts · lib/date.ts
Daily priority / Top 3	An owner/date/position 1–3 row normally linked to a canonical task.	db/schema/daily-priorities.ts · app/(app)/today/**
Auto goal progress	Completed direct tasks divided by non-cancelled direct tasks; completed goal forced to 100%.	lib/goal-progress.ts
Habit outcome	Derived done/partial/missed/skipped/pending/off-schedule meaning from schedule, raw status, value, target, and direction.	lib/habit-streaks.ts · lib/habit-view.ts
Focus credited time	Actual elapsed seconds with pause handling, capped to planned duration.	lib/focus.ts · lib/focus-reconcile.ts
Weekly Review	Stored reflection attached to a live derivation of that week's canonical activity.	lib/review.ts · db/schema/reviews.ts
Task Map	Persisted viewport/legend plus semantic nodes/edges, optionally linked to canonical tasks.	db/schema/task-maps.ts · components/task-maps/**
Archive	Soft-hidden state for Life Areas, Goals, Habits, and Maps; restored centrally in Settings.	app/(app)/settings/archive-actions.ts
Revalidation	Next path refresh requested after a Server Action; explicit downstream coverage is incomplete.	app/(app)/**/actions.ts
RLS	PostgreSQL Row Level Security. It is not configured in current migrations.	db/migrations/**

Final audit statement

This document captures the entire inspectable GoHa application system at the audited local snapshot: product behavior, route shell, implementation architecture, database and migration model, validation and lifecycle semantics, authentication and security posture, visual/accessibility system, test/operations state, prioritized risks, and every tracked/relevant untracked repository path. Runtime infrastructure and live data remain outside the read-only boundary and are explicitly identified wherever their state would change a conclusion.

Suggested next use

Use this report as the baseline for a clean CURRENT_STATE document and issue backlog. Resolve P0 items before deployment/test mutation, then convert each high/medium finding into an owned, testable change with an evidence link and acceptance criteria.